

SELENIUM WITH JAVA

Complete Master Program

From Zero to Expert — Every Method, Every Concept, Every Example

What You Will Learn

- Module 1 — What is Selenium? Architecture, Setup & First Script
- Module 2 — Locating Web Elements: All 8 Strategies with Deep Examples
- Module 3 — Every Browser & Element Interaction Method Explained
- Module 4 — Waits & Synchronization — The Most Critical Skill
- Module 5 — Actions Class, Keyboard, Mouse, Drag & Drop, Scroll
- Module 6 — JavaScript Executor & Screenshots
- Module 7 — TestNG Framework — Annotations, Assertions, DataProvider
- Module 8 — Page Object Model (POM) — Professional Project Structure
- Module 9 — Data-Driven Testing, Configuration & Reporting
- Module 10 — Advanced: Grid, Cross-Browser, CI/CD, Cucumber BDD
- Module 11 — Best Practices, Common Errors & Debugging Guide

Prerequisite: Java knowledge (OOP, Collections, Exception Handling)

MODULE 1 What is Selenium? Architecture, Setup & First Script

Module 1: What is Selenium?

If you are completely new to Selenium, this is where your journey begins. We will cover exactly what Selenium is, how it works under the hood, and get your development environment fully set up so you can run your very first automated browser script.

1.1 What is Selenium? — The Big Picture

Imagine you are testing a login page manually. You open the browser, type a username, type a password, click Login, and verify that the dashboard appears. Now imagine having to do this 500 times a day, across Chrome, Firefox, and Edge, on Windows and Mac simultaneously. That is impossible to do manually. Selenium automates it for you.

Selenium is a free, open-source automation framework that controls a web browser exactly the way a human would — clicking buttons, typing text, reading content — but at machine speed, any number of times, without fatigue.

Selenium is NOT:

- A testing framework on its own (we pair it with TestNG)
- A tool for testing desktop applications (browsers only)
- A performance testing tool (use JMeter for that)

Selenium IS:

- The industry-standard tool for web browser automation
- Used by companies like Google, Netflix, and Amazon for testing
- Free and works with Java, Python, C#, Ruby, JavaScript

1.2 The Selenium Suite — Three Parts

Component	What It Does
<code>Selenium WebDriver</code>	The core API — controls the browser directly from your Java code. This is what we use 95% of the time.
<code>Selenium IDE</code>	A browser extension for record & playback. Good for demos, not real projects.
<code>Selenium Grid</code>	Runs tests on multiple machines/browsers in parallel. Used in advanced setups.

1.3 How Selenium WebDriver Works — Architecture

Before you write a single line of code, understand HOW it works. This knowledge will save you hours of debugging later.

When you write `driver.findElement(By.id("btn")).click()` in Java, here is the exact chain of events:

1. Your Java code calls the Selenium WebDriver API
2. The API converts your command into a JSON message (W3C WebDriver Protocol)
3. The JSON is sent over HTTP to the Browser Driver (e.g., ChromeDriver)
4. ChromeDriver translates the JSON into Chrome's native DevTools commands
5. Chrome performs the action (clicks the button)
6. Chrome reports back the result, which travels the same chain in reverse
7. Your Java code receives the result (success or exception)

This is why you need BOTH the Selenium library AND a browser driver — they are two separate things working together.

Layer	What It Is	Example
Your Java Code	Selenium WebDriver API calls	<code>driver.get(url)</code>
Selenium Library	The .jar file added via Maven	<code>selenium-java-4.x.jar</code>
Browser Driver	Separate executable, browser-specific	<code>chromedriver.exe</code>
Browser	The actual browser being controlled	Google Chrome

1.4 Setting Up Your Development Environment

We will use Maven to manage dependencies — no manual JAR downloading.

Step 1: Install Prerequisites

- Java JDK 11 or higher — download from adoptium.net
- IntelliJ IDEA (recommended) or Eclipse
- Google Chrome browser (latest version)

Step 2: Create a Maven Project in IntelliJ

8. Open IntelliJ → File → New → Project
9. Select Maven, choose Java 11+, click Next
10. Name your project: SeleniumMasterProgram

11. Click Finish

Step 3: Add Dependencies to pom.xml

Open pom.xml and add these dependencies inside the <dependencies> tag:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.selenium.master</groupId>
  <artifactId>SeleniumMasterProgram</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>

  <dependencies>

    <!-- Selenium Java – the core library -->
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.18.1</version>
    </dependency>

    <!-- WebDriverManager – auto downloads ChromeDriver for you -->
    <!-- Without this, you'd manually download and manage drivers -->
    <dependency>
      <groupId>io.github.bonigarcia</groupId>
      <artifactId>webdrivermanager</artifactId>
      <version>5.7.0</version>
    </dependency>

    <!-- TestNG – test framework (like JUnit but more powerful) -->
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.9.0</version>
      <scope>test</scope>
    </dependency>

    <!-- Apache Commons IO – for screenshot saving -->
    <dependency>
      <groupId>commons-io</groupId>
      <artifactId>commons-io</artifactId>
      <version>2.15.1</version>
    </dependency>

  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.2</version>
      </plugin>
```

```

    </plugins>
</build>

</project>

```

NOTE: After saving `pom.xml`, IntelliJ will auto-download all dependencies. Wait for it to finish (watch the progress bar at the bottom).

1.5 Your First Selenium Script — Line by Line

Create a new Java class: `src/main/java/FirstTest.java`

Example: Open Google, print the title, then close the browser

```

import io.github.bonigarcia.wdm.WebDriverManager;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class FirstTest {

    public static void main(String[] args) {

        // — STEP 1: Setup ChromeDriver automatically —————
        // WebDriverManager checks your Chrome version and downloads
        // the matching ChromeDriver binary. No manual downloads!
        WebDriverManager.chromedriver().setup();

        // — STEP 2: Create WebDriver instance —————
        // WebDriver is an interface. ChromeDriver is one implementation.
        // This line opens a new Chrome browser window.
        WebDriver driver = new ChromeDriver();

        // — STEP 3: Maximize the browser window —————
        // Without this, Chrome opens in a small window.
        // Some elements may not be visible in a small window.
        driver.manage().window().maximize();

        // — STEP 4: Navigate to a URL —————
        // driver.get() opens the specified URL and WAITS for the
        // page to fully load before your next line runs.
        driver.get("https://www.google.com");

        // — STEP 5: Get the page title —————
        // Returns the text in the browser's title bar / tab.
        String pageTitle = driver.getTitle();
        System.out.println("Page Title is: " + pageTitle);
        // Output: Page Title is: Google

        // — STEP 6: Get the current URL —————
        String currentUrl = driver.getCurrentUrl();
        System.out.println("Current URL: " + currentUrl);
        // Output: Current URL: https://www.google.com/

        // — STEP 7: Close the browser —————
        // ALWAYS call quit() - not close().
        // quit() = closes browser + kills driver process
        // close() = closes ONLY the current window (driver still runs)
        driver.quit();
        System.out.println("Browser closed. Test done!");
    }
}

```

}

WARNING: If you forget `driver.quit()`, the `ChromeDriver` process keeps running in the background and eats memory. Always quit in a finally block in real tests.

Example: Navigate to multiple pages in sequence

```
WebDriverManager.chromedriver().setup();
WebDriver driver = new ChromeDriver();
driver.manage().window().maximize();

// Navigate to first URL
driver.get("https://www.google.com");
System.out.println("1. Title: " + driver.getTitle());

// Navigate to second URL — replaces the first
driver.get("https://www.github.com");
System.out.println("2. Title: " + driver.getTitle());

// Navigate to third URL
driver.get("https://www.wikipedia.org");
System.out.println("3. Title: " + driver.getTitle());

// Go back to GitHub (like clicking browser's Back button)
driver.navigate().back();
System.out.println("After back: " + driver.getTitle());

// Go forward to Wikipedia
driver.navigate().forward();
System.out.println("After forward: " + driver.getTitle());

// Refresh current page
driver.navigate().refresh();
System.out.println("After refresh: " + driver.getTitle());

driver.quit();
```

TIP: `driver.get()` and `driver.navigate().to()` do the same thing. Use `driver.get()` for the initial navigation and `driver.navigate()` for back/forward/refresh.

1.6 WebDriver Methods — Every Navigation Method Explained

Method	What It Does
<code>driver.get(url)</code>	Opens a URL. Waits for page to fully load.
<code>driver.navigate().to(url)</code>	Same as <code>get()</code> — alternate syntax for navigation.
<code>driver.navigate().back()</code>	Goes to the previous page (browser Back button).
<code>driver.navigate().forward()</code>	Goes to the next page (browser Forward button).
<code>driver.navigate().refresh()</code>	Reloads the current page (like pressing F5).
<code>driver.getTitle()</code>	Returns the page title as a String.
<code>driver.getCurrentUrl()</code>	Returns the full current URL as a String.
<code>driver.getPageSource()</code>	Returns the entire HTML source of the current page.
<code>driver.close()</code>	Closes the current browser window only.

<code>driver.quit()</code>	Closes ALL browser windows and kills the driver process.
<code>driver.manage().window().maximize()</code>	Maximizes the browser window to full screen.
<code>driver.manage().window().minimize()</code>	Minimizes the browser window to taskbar.
<code>driver.manage().window().fullscreen()</code>	Makes browser go into fullscreen (F11 mode).

MODULE 2 Locating Web Elements — All 8 Strategies

Module 2: Locating Web Elements

This is the most fundamental skill in Selenium. Everything you do — clicking, typing, reading text — starts with finding the right element on the page. Selenium gives you 8 different strategies to locate elements. We will master every single one of them.

2.1 What is a Web Element?

Every visible thing on a webpage — a button, a text box, a link, an image, a dropdown — is a "web element". In HTML, elements are defined by tags like `<input>`, `<button>`, `<a>`, `<div>`, etc. Each element can have attributes like `id`, `name`, `class`, `type`, `href`, and more.

When you inspect a page (press F12 in Chrome → Elements tab), you see the raw HTML. For example:

```
<!-- A login form in HTML -->
<form id="loginForm">
  <input id="username" name="user" class="form-control" type="text" placeholder="Enter
username" />
  <input id="password" name="pass" class="form-control" type="password" />
  <button id="loginBtn" class="btn btn-primary" type="submit">Login</button>
  <a href="/forgot" class="link">Forgot Password?</a>
</form>
```

From this HTML, you can locate each element using any of its attributes. Let's go through all 8 strategies.

2.2 Strategy 1 — By.id()

The `id` attribute is supposed to be **UNIQUE** on a page. If an element has an `id`, use it — it is the fastest and most reliable locator.

Example: Locate by id — the simplest approach

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;

// Finds: <input id="username" ... />
WebElement usernameField = driver.findElement(By.id("username"));

// Finds: <button id="loginBtn" ...>Login</button>
WebElement loginButton = driver.findElement(By.id("loginBtn"));

// Finds: <input id="password" ... />
WebElement passwordField = driver.findElement(By.id("password"));

// Now interact with them:
```



```
usernameField.sendKeys("john.doe");
passwordField.sendKeys("mypassword");
loginButton.click();
```

TIP: Always check the id first when inspecting an element. If it has an id and it looks unique (not auto-generated like 'input_1234'), use By.id().

2.3 Strategy 2 — By.name()

The name attribute is commonly used on form fields (input, select, textarea). If there is no id, name is your next best option for form elements.

Example: Locate form fields using name attribute

```
// Finds: <input name="user" ... />
WebElement username = driver.findElement(By.name("user"));

// Finds: <input name="pass" ... />
WebElement password = driver.findElement(By.name("pass"));

username.sendKeys("testuser");
password.sendKeys("testpass");
```

2.4 Strategy 3 — By.className()

Locates elements by their CSS class name. Note: if an element has multiple classes like class="btn btn-primary", you can only use ONE class name with By.className().

Example: Locate by class name — single class

```
// Finds: <button class="btn btn-primary">Login</button>
// Use just ONE class name:
WebElement btn = driver.findElement(By.className("btn-primary"));

// Finds: <input class="form-control" .../>
WebElement input = driver.findElement(By.className("form-control"));

// Find ALL elements with a class - returns a List:
List<WebElement> allFormControls = driver.findElements(By.className("form-control"));
System.out.println("Number of form fields: " + allFormControls.size());
```

WARNING: *By.className() fails if you pass multiple classes like "btn btn-primary". For multi-class matching, use By.cssSelector() instead.*

2.5 Strategy 4 — By.tagName()

Locates elements by their HTML tag type. Since most pages have many elements of the same tag, this is most useful with findElements() to get ALL of them.

Example: Get all links, all inputs, all table rows on a page

```
import java.util.List;

// Get ALL anchor tags (links) on the page
List<WebElement> allLinks = driver.findElements(By.tagName("a"));
System.out.println("Total links on page: " + allLinks.size());

// Print every link's text and href
for (WebElement link : allLinks) {
    System.out.println(link.getText() + " -> " + link.getAttribute("href"));
}

// Get ALL input fields on a form
List<WebElement> inputs = driver.findElements(By.tagName("input"));
System.out.println("Total input fields: " + inputs.size());

// Get ALL table rows
List<WebElement> rows = driver.findElements(By.tagName("tr"));
System.out.println("Table has " + rows.size() + " rows");
```

2.6 Strategy 5 — By.linkText()

Locates anchor (<a>) elements by their exact visible text. The match must be EXACT — case-sensitive, including spaces.

Example: Locate links by their exact text

```
// Finds: <a href="/forgot">Forgot Password?</a>
// The text MUST match exactly — every character, every space
WebElement forgotLink = driver.findElement(By.linkText("Forgot Password?"));
forgotLink.click();

// Finds: <a href="/register">Create New Account</a>
WebElement registerLink = driver.findElement(By.linkText("Create New Account"));

// This WOULD FAIL because the text is wrong:
// driver.findElement(By.linkText("forgot password")); // case mismatch!
// driver.findElement(By.linkText("Forgot Password")); // missing '?'
```

2.7 Strategy 6 — By.partialLinkText()

Like By.linkText() but matches any anchor element whose text CONTAINS your string. Useful when the link text is long or dynamic.

Example: Locate links using partial text

```
// Finds any link containing the word "Forgot"
WebElement forgotLink = driver.findElement(By.partialLinkText("Forgot"));

// Useful for dynamic links like:
// <a href="/order/123">View Order #123 - Placed on 15 May 2026</a>
// You can match on just the stable part:
WebElement orderLink = driver.findElement(By.partialLinkText("View Order"));

// Find ALL links that contain "Sign"
// Matches: "Sign In", "Sign Up", "Sign Out"
```

```
List<WebElement> signLinks = driver.findElements(By.partialLinkText("Sign"));
for (WebElement link : signLinks) {
    System.out.println("Found: " + link.getText());
}
```

2.8 Strategy 7 — By.cssSelector() — Fast & Powerful

CSS Selectors are the same selectors used in CSS stylesheets. They are faster than XPath and very readable. This is one of the two most important locator strategies you must master.

CSS Selector Syntax Guide

Selector Pattern	What It Matches
#elementId	Element with id="elementId"
.className	Element with class containing className
tagName	All elements of that tag type
tag.class	Elements of that tag AND that class
tag#id	Element of that tag AND that id
[attribute='value']	Element with that attribute=value
[attribute*='value']	Attribute CONTAINS value
[attribute^='value']	Attribute STARTS WITH value
[attribute\$='value']	Attribute ENDS WITH value
parent > child	Direct child elements only
ancestor descendant	All descendants (any depth)
tag:nth-child(n)	nth child of its parent
tag:first-child	First child of its parent
tag:last-child	Last child of its parent

Example: CSS Selector examples — all common patterns

```
// — By ID —————
// Matches: <input id="username">
driver.findElement(By.cssSelector("#username"));

// — By Class —————
// Matches: <button class="btn-primary">
driver.findElement(By.cssSelector(".btn-primary"));

// — By Tag + ID —————
// More specific: ONLY input tag with that id
driver.findElement(By.cssSelector("input#username"));

// — By Tag + Class —————
// Matches: <button class="btn btn-primary">
driver.findElement(By.cssSelector("button.btn-primary"));
```

```

// — By Multiple Classes —————
// Matches elements with BOTH classes
driver.findElement(By.cssSelector(".btn.btn-primary"));

// — By Attribute —————
// Matches: <input type="password">
driver.findElement(By.cssSelector("input[type='password']"));

// Matches: <input placeholder="Enter email">
driver.findElement(By.cssSelector("input[placeholder='Enter email']"));

// — Attribute Contains (*=) —————
// Matches href that contains 'login' anywhere
driver.findElement(By.cssSelector("a[href*='login']"));

// — Attribute Starts With (^=) —————
// Matches href that starts with 'https'
driver.findElements(By.cssSelector("a[href^='https']"));

// — Attribute Ends With ($=) —————
// Matches image src ending in .png
driver.findElements(By.cssSelector("img[src$='.png']"));

// — Direct Child (>) —————
// Matches input that is a DIRECT child of div#loginForm
driver.findElement(By.cssSelector("div#loginForm > input"));

// — Any Descendant (space) —————
// Matches any input inside div#loginForm at any depth
driver.findElements(By.cssSelector("div#loginForm input"));

// — nth-child —————
// Matches the 3rd row in a table
driver.findElement(By.cssSelector("table tr:nth-child(3)"));

// Matches the 2nd td in the 3rd row
driver.findElement(By.cssSelector("table tr:nth-child(3) td:nth-child(2)"));

// — Combined —————
// Complex selector: submit button inside a form
driver.findElement(By.cssSelector("form#loginForm button[type='submit']"));

```

2.9 Strategy 8 — By.xpath() — The Ultimate Power Tool

XPath (XML Path Language) is the most powerful and flexible locator. It can find ANY element using its tag, attributes, text content, position, or relationship to other elements. When all other locators fail, XPath works.

XPath Syntax Fundamentals

XPath Expression	What It Means
//tag	Find tag anywhere in the document
//tag[@attr='value']	Find tag where attribute equals value
//tag[text()='exact text']	Find tag with exact inner text

<code>//tag[contains(@attr, 'part')]</code>	Attribute CONTAINS a substring
<code>//tag[contains(text(), 'part')]</code>	Text CONTAINS a substring
<code>//tag[starts-with(@attr, 'val')]</code>	Attribute STARTS WITH a substring
<code>(//tag[@attr='val'])[2]</code>	The SECOND matching element (1-indexed)
<code>//parent/child</code>	Direct child of parent
<code>//ancestor//descendant</code>	Descendant at any depth

Example: XPath — Basic patterns you will use every day

```
// — By Attribute —————
// Matches: <input id="username">
driver.findElement(By.xpath("//input[@id='username']"));

// Matches: <button type="submit">
driver.findElement(By.xpath("//button[@type='submit']"));

// Matches: <input name="email" type="text">
driver.findElement(By.xpath("//input[@name='email'][@type='text']"));

// — By Exact Text —————
// Matches: <a>Forgot Password?</a>
driver.findElement(By.xpath("//a[text()='Forgot Password?']"));

// Matches: <button>Login</button>
driver.findElement(By.xpath("//button[text()='Login']"));

// — Contains —————
// Matches any a whose text contains 'Forgot'
driver.findElement(By.xpath("//a[contains(text(), 'Forgot')]"));

// Matches any element whose class contains 'btn-primary'
driver.findElement(By.xpath("//*[contains(@class, 'btn-primary')]"));

// — Starts-with —————
// Useful for auto-generated IDs like 'user_12345'
driver.findElement(By.xpath("//input[starts-with(@id, 'user_')]"));

// — Index — Select nth match —————
// Page has multiple <input type='text'>, get the 2nd one
driver.findElement(By.xpath("//input[@type='text'][2]"));

// Get the last matching element
driver.findElement(By.xpath("//tr[last()]"));

// Get second to last
driver.findElement(By.xpath("//tr[last()-1]"));
```

Example: XPath Axes — navigating DOM relationships

```
// — parent:: —————
// Find the parent <div> of an input
// Useful when the input has no useful attributes but its parent does
driver.findElement(By.xpath("//input[@id='username']/parent::div"));

// — ancestor:: —————
```

```
// Find any ancestor <form> of an element
driver.findElement(By.xpath("//input[@id='username']/ancestor::form"));

// — following-sibling:: —————
// HTML: <label>Email</label><input type='text'>
// Find the input that follows the label with text 'Email'
driver.findElement(By.xpath("//label[text()='Email']/following-sibling::input"));

// — preceding-sibling:: —————
// Find the label that comes BEFORE an input
driver.findElement(By.xpath("//input[@id='email']/preceding-sibling::label"));

// — child:: —————
// Find all direct li children of a ul
driver.findElements(By.xpath("//ul[@id='navMenu']/child::li"));

// — descendant:: —————
// Find ALL buttons inside a form, at any depth
driver.findElements(By.xpath("//form[@id='loginForm']/descendant::button"));

// — REAL WORLD EXAMPLE —————
// HTML Table: Find a row where first column says 'John', then click Edit
// <table><tr><td>John</td><td>Doe</td><td><a>Edit</a></td></tr></table>
driver.findElement(By.xpath(
    "//td[text()='John']/following-sibling::td/a[text()='Edit']"
)).click();
```

TIP: To test your XPath in Chrome: Press F12, go to Console tab, type `$x("//your/xpath")` and press Enter. It shows matching elements instantly.

2.10 findElement() vs findElements()

These are the two core methods for locating elements. Understand their differences well.

Aspect	findElement()	findElements()
Returns	Single WebElement	List<WebElement>
If NOT found	Throws NoSuchElementException	Returns empty List []
If MULTIPLE match	Returns first match	Returns all matches
Use when	You expect exactly one element	You expect zero or more elements

Example: Using both findElement and findElements correctly

```
// — findElement: expects exactly one match —————
WebElement loginBtn = driver.findElement(By.id("loginBtn"));
// If not found: throws NoSuchElementException (test fails)

// — findElements: safe — never throws exception —————
List<WebElement> links = driver.findElements(By.tagName("a"));
// If none found: returns empty list, no exception

// — How to CHECK if an element EXISTS safely —————
// WRONG way — throws exception if not found:
// boolean exists = driver.findElement(By.id("errorMsg")).isDisplayed();
```

```
// CORRECT way — use findElements and check size:
boolean errorExists = !driver.findElements(By.id("errorMsg")).isEmpty();
if (errorExists) {
    System.out.println("Error message is present!");
}

// — Iterating findElements results —————
List<WebElement> tableRows = driver.findElements(By.tagName("tr"));
System.out.println("Table has " + tableRows.size() + " rows");

for (int i = 0; i < tableRows.size(); i++) {
    System.out.println("Row " + (i+1) + ": " + tableRows.get(i).getText());
}

// — Get a specific item from the list —————
WebElement firstRow = tableRows.get(0); // index 0 = first element
WebElement lastRow = tableRows.get(tableRows.size() - 1);
```

2.11 Locator Priority — Which to Use When

When you inspect an element and decide which locator to use, follow this priority order:

12. By.id() — if the element has a unique, stable id
13. By.cssSelector() — for most other cases, great balance of speed and flexibility
14. By.name() — for form fields that have a name attribute
15. By.linkText() or By.partialLinkText() — only for anchor links
16. By.className() — only if class is unique enough
17. By.xpath() — when CSS cannot express the relationship (like navigating to siblings/parents)
18. By.tagName() — only when you want ALL elements of that type

WARNING: Avoid auto-generated IDs like `id="input_1234567"` — they change every page load and make your tests flaky. Look for stable attributes instead.

2.12 Real-World Locator Challenges — Practice Scenarios

These are the kinds of HTML you will actually encounter in real projects. Study each scenario.

Scenario A: Dynamic Table — Find and Click Edit for a Specific Row

```
// HTML Structure:
// <table id='usersTable'>
//   <tr><th>Name</th><th>Email</th><th>Action</th></tr>
//   <tr><td>Alice</td><td>alice@co.com</td><td><button class='edit-
btn'>Edit</button></td></tr>
//   <tr><td>Bob</td>   <td>bob@co.com</td>   <td><button class='edit-
btn'>Edit</button></td></tr>
//   <tr><td>Carol</td><td>carol@co.com</td><td><button class='edit-
btn'>Edit</button></td></tr>
// </table>

// GOAL: Click the Edit button on Bob's row WITHOUT knowing its row index

// XPath approach — find td with text 'Bob', navigate to sibling td, find button
driver.findElement(By.xpath(
    "//td[text()='Bob']/following-sibling::td/button[@class='edit-btn']"
)).click();

// Alternative: find the row that contains 'Bob', then find button within it
driver.findElement(By.xpath(
    "//tr[td[text()='Bob']]//button[@class='edit-btn']"
)).click();

// CSS approach — not possible to match by text content, XPath is better here
```

Scenario B: Read All Rows From a Table Into a List

```
// Get all data rows (skip the header row at index 0)
List<WebElement> rows = driver.findElements(
    By.cssSelector("#usersTable tr:not(:first-child)")
);

System.out.println("Users in table: " + rows.size());

for (WebElement row : rows) {
    // Get all cells in this row
    List<WebElement> cells = row.findElements(By.tagName("td"));
    // Note: findElements on an ELEMENT searches within that element only

    String name = cells.get(0).getText();
    String email = cells.get(1).getText();
    System.out.printf("Name: %-15s Email: %s\n", name, email);
}

// Output:
// Users in table: 3
// Name: Alice           Email: alice@co.com
// Name: Bob             Email: bob@co.com
// Name: Carol           Email: carol@co.com
```

Scenario C: Element Has No Unique ID or Name — Use Context


```
// HTML: A form with labels next to inputs, no id attributes
// <form>
//   <div class='field'>
//     <label>First Name</label>
//     <input type='text' placeholder='Enter first name'>
//   </div>
//   <div class='field'>
//     <label>Last Name</label>
//     <input type='text' placeholder='Enter last name'>
//   </div>
// </form>

// Strategy: find the label, navigate to sibling input
driver.findElement(By.xpath(
    "//label[text()='First Name']/following-sibling::input"
)).sendKeys("John");

driver.findElement(By.xpath(
    "//label[text()='Last Name']/following-sibling::input"
)).sendKeys("Doe");

// Alternative: find the parent div, then the input inside it
driver.findElement(By.xpath(
    "//div[@class='field'][label[text()='First Name']]/input"
)).sendKeys("John");
```

Scenario D: Autocomplete / Suggestion Dropdown

```
// HTML: <input id='citySearch'> → triggers a suggestion list as you type
// <ul class='suggestions'>
//   <li>Chennai</li>
//   <li>Chandigarh</li>
//   <li>Chicago</li>
// </ul>

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

// Step 1: Type into the autocomplete field
driver.findElement(By.id("citySearch")).sendKeys("Chen");

// Step 2: Wait for the suggestion list to appear
wait.until(ExpectedConditions.visibilityOfElementLocated(
    By.cssSelector("ul.suggestions")
));

// Step 3: Find ALL suggestions
List<WebElement> suggestions = driver.findElements(
    By.cssSelector("ul.suggestions li")
);

// Step 4: Click the one you want
for (WebElement suggestion : suggestions) {
    if (suggestion.getText().equals("Chennai")) {
        suggestion.click();
        break;
    }
}

// Verify the field now has the selected value
String selected = driver.findElement(By.id("citySearch")).getAttribute("value");
Assert.assertEquals(selected, "Chennai");
```

Scenario E: File Upload

```
// HTML: <input type='file' id='fileUpload'>
// You cannot click this with normal Selenium and browse to a file.
// Instead: directly send the file path using sendKeys()

WebElement fileInput = driver.findElement(By.id("fileUpload"));

// Send the ABSOLUTE path of the file you want to upload
fileInput.sendKeys("/home/user/documents/testfile.pdf");

// On Windows use double backslash or forward slash:
fileInput.sendKeys("C:\\Users\\YourName\\Desktop\\testfile.pdf");
// OR: fileInput.sendKeys("C:/Users/YourName/Desktop/testfile.pdf");

// Wait for the filename to appear in the UI, then submit
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.attributeContains(
    By.id("fileUpload"), "value", "testfile.pdf"
));

driver.findElement(By.id("uploadBtn")).click();
```

NOTE: The file input `sendKeys()` approach works **ONLY** when the `<input type='file'>` element is actually present in the DOM. If it is hidden, use `JavascriptExecutor` to make it visible first.

MODULE 3 Every Browser & Element Interaction Method

Module 3: Interacting with Web Elements

Now that you can find elements, you need to interact with them. Selenium provides a rich set of methods for every kind of interaction — typing, clicking, reading text, checking state, handling forms, alerts, frames, and browser windows.

3.1 WebElement Methods — Every Method Explained

Method	What It Does
<code>element.click()</code>	Simulates a mouse click on the element.
<code>element.sendKeys("text")</code>	Types text into an input/textarea field.
<code>element.clear()</code>	Clears the current value of an input field.
<code>element.getText()</code>	Returns the visible text inside the element.
<code>element.getAttribute("attrName")</code>	Returns the value of an HTML attribute (id, class, href, value, etc).
<code>element.getCssValue("prop")</code>	Returns the computed CSS value (color, font-size, etc).
<code>element.isDisplayed()</code>	Returns true if element is visible on screen.
<code>element.isEnabled()</code>	Returns true if element is not disabled.
<code>element.isSelected()</code>	Returns true if checkbox/radio is checked.
<code>element.submit()</code>	Submits a form (same as clicking the submit button).
<code>element.getTagName()</code>	Returns the HTML tag name (input, button, a, etc).
<code>element.getSize()</code>	Returns the width and height as a Dimension object.
<code>element.getLocation()</code>	Returns the x,y coordinates of the element as a Point.

Example: Using every WebElement method

```
WebElement usernameField = driver.findElement(By.id("username"));

// — Typing text —————
usernameField.sendKeys("john.doe@company.com");

// — Clear the field then type new text —————
// Always clear() before sendKeys() if the field might already have text
usernameField.clear();
usernameField.sendKeys("new.email@company.com");

// — Read the current value of an input —————
// getText() returns empty string for inputs — use getAttribute("value")
String currentValue = usernameField.getAttribute("value");
```

```

System.out.println("Field contains: " + currentValue);

// — Get visible text of a non-input element —————
WebElement heading = driver.findElement(By.tagName("h1"));
String headingText = heading.getText();
System.out.println("Page heading: " + headingText);

// — Get any attribute —————
WebElement link = driver.findElement(By.tagName("a"));
System.out.println("href: " + link.getAttribute("href"));
System.out.println("class: " + link.getAttribute("class"));
System.out.println("id: " + link.getAttribute("id"));

// — Check element state —————
WebElement submitBtn = driver.findElement(By.id("submitBtn"));
System.out.println("Visible: " + submitBtn.isDisplayed()); // true/false
System.out.println("Enabled: " + submitBtn.isEnabled()); // true/false

WebElement checkbox = driver.findElement(By.id("agreeCheckbox"));
System.out.println("Checked: " + checkbox.isSelected()); // true/false

// — Get CSS property —————
String bgColor = submitBtn.getCssValue("background-color");
String fontSize = submitBtn.getCssValue("font-size");
System.out.println("Button color: " + bgColor);

// — Dimensions and position —————
System.out.println("Width: " + submitBtn.getSize().getWidth());
System.out.println("Height: " + submitBtn.getSize().getHeight());
System.out.println("X pos: " + submitBtn.getLocation().getX());
System.out.println("Y pos: " + submitBtn.getLocation().getY());

```

3.2 Handling Dropdowns with the Select Class

Standard HTML `<select>` dropdown elements need the special `Select` class. You cannot just click options directly — you must wrap the element in a `Select` object first.

Here is the HTML we will work with:

```

<!-- HTML of a country dropdown -->
<select id="country" name="country">
  <option value="">-- Select Country --</option>
  <option value="IN">India</option>
  <option value="US">United States</option>
  <option value="UK">United Kingdom</option>
  <option value="AU">Australia</option>
</select>

```

Example: All three ways to select a dropdown option

```

import org.openqa.selenium.support.ui.Select;

// Step 1: Find the <select> element
WebElement dropdownElement = driver.findElement(By.id("country"));

// Step 2: Wrap it in Select object
Select select = new Select(dropdownElement);

// — Method 1: By visible text (what the user sees) —————

```

```
// Selects the option that displays "India"
select.selectByVisibleText("India");

// — Method 2: By value attribute —————
// Selects: <option value="US">United States</option>
select.selectByValue("US");

// — Method 3: By index (0-based) —————
// Selects the 3rd option (index 2): United Kingdom
// Index 0 = '-- Select Country --'
// Index 1 = 'India'
// Index 2 = 'United States'
select.selectByIndex(2);

// — Reading the selected value —————
String selectedText = select.getFirstSelectedOption().getText();
System.out.println("You selected: " + selectedText);

// — Get all options in the dropdown —————
List<WebElement> allOptions = select.getOptions();
System.out.println("Dropdown has " + allOptions.size() + " options:");
for (WebElement option : allOptions) {
    System.out.println(" - " + option.getText()
        + " (value=" + option.getAttribute("value") + ")");
}
```

Example: Multi-select dropdown — select and deselect multiple options

```
// HTML: <select id="hobbies" multiple>
//         <option value="read">Reading</option>
//         <option value="code">Coding</option>
//         <option value="music">Music</option>
//         </select>

Select multiSelect = new Select(driver.findElement(By.id("hobbies")));

// Check if it IS a multi-select before trying
if (multiSelect.isMultiple()) {

    // Select multiple options
    multiSelect.selectByVisibleText("Reading");
    multiSelect.selectByVisibleText("Coding");
    multiSelect.selectByValue("music");

    // Read all currently selected options
    List<WebElement> selected = multiSelect.getAllSelectedOptions();
    System.out.println("Selected " + selected.size() + " options:");
    for (WebElement opt : selected) {
        System.out.println("  " + opt.getText());
    }

    // Deselect a specific option
    multiSelect.deselectByVisibleText("Reading");

    // Deselect ALL selected options
    multiSelect.deselectAll();
}
```

3.3 Checkboxes and Radio Buttons

Checkboxes and radio buttons are just regular `<input>` elements. The key is to always check the current state before clicking — you should never blindly click without knowing if it's already in the desired state.

Example: Handling checkboxes — check, uncheck, verify

```
// HTML: <input type="checkbox" id="agree" name="terms">

WebElement checkbox = driver.findElement(By.id("agree"));

// — Check the current state first —————
System.out.println("Currently checked: " + checkbox.isSelected());

// — Only check if it is NOT already checked —————
if (!checkbox.isSelected()) {
    checkbox.click();
    System.out.println("Checkbox is now checked.");
}

// — Only uncheck if it IS currently checked —————
if (checkbox.isSelected()) {
    checkbox.click();
    System.out.println("Checkbox is now unchecked.");
}

// — Verify checkbox is checked after action —————
checkbox.click();
Assert.assertTrue(checkbox.isSelected(), "Checkbox should be checked!");
```

Example: Handling radio buttons — selecting from a group

```
// HTML: <input type="radio" name="gender" value="male"> Male
//        <input type="radio" name="gender" value="female"> Female
//        <input type="radio" name="gender" value="other"> Other

// — Method 1: Find by value attribute —————
driver.findElement(By.cssSelector("input[type='radio'][value='female']")).click();

// — Method 2: Find all in group, select the matching one —————
List<WebElement> radios = driver.findElements(By.name("gender"));

for (WebElement radio : radios) {
    if (radio.getAttribute("value").equals("male")) {
        radio.click();
        break; // Stop after selecting
    }
}

// — Verify which radio is selected —————
for (WebElement radio : radios) {
    if (radio.isSelected()) {
        System.out.println("Selected gender: " + radio.getAttribute("value"));
    }
}
```

3.4 Handling Browser Alerts, Confirms & Prompts

Browser alerts are popup dialog boxes created by JavaScript. They appear on top of the page and BLOCK all interaction until handled. There are three types:

Alert Type	Description
Alert	Simple message with OK button only. <code>driver.accept()</code> to close.
Confirm	Message with OK and Cancel. <code>accept()</code> = OK, <code>dismiss()</code> = Cancel.
Prompt	Message with a text input field + OK/Cancel. Use <code>sendKeys()</code> to type.

Example: Handling all three alert types

```
import org.openqa.selenium.Alert;

// — ALERT — simple OK popup —————
// Trigger: driver.findElement(By.id("triggerAlert")).click();

// Wait for alert to appear, then switch to it
Alert alert = driver.switchTo().alert();

// Read the alert message
String alertMessage = alert.getText();
System.out.println("Alert says: " + alertMessage);

// Accept (click OK)
alert.accept();
System.out.println("Alert dismissed.");

// — CONFIRM — OK or Cancel —————
// Trigger: driver.findElement(By.id("triggerConfirm")).click();

Alert confirm = driver.switchTo().alert();
System.out.println("Confirm says: " + confirm.getText());

// Click OK
confirm.accept();

// OR click Cancel:
// confirm.dismiss();

// — PROMPT — alert with text input —————
// Trigger: driver.findElement(By.id("triggerPrompt")).click();

Alert prompt = driver.switchTo().alert();
System.out.println("Prompt says: " + prompt.getText());

// Type text into the prompt input
prompt.sendKeys("John Doe");

// Accept (click OK with the typed text)
prompt.accept();

// — Using Explicit Wait for Alert —————
// Sometimes alert takes a moment to appear. Use wait:
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
Alert safeAlert = wait.until(ExpectedConditions.alertIsPresent());
safeAlert.accept();
```

WARNING: Never try to `findElement()` while an alert is open. The browser blocks all DOM interaction until the alert is handled. Switch to the alert, handle it, then continue.

3.5 Handling iFrames

An iFrame (inline frame) is an HTML element that embeds another webpage inside the current page. Elements inside an iFrame are NOT accessible from the main page context. You must explicitly switch into the iFrame first.

Example: All three ways to switch to an iFrame

```
// — Method 1: Switch by index (0 = first iFrame on page) —————
driver.switchTo().frame(0);

// Now you can interact with elements INSIDE the first iFrame
driver.findElement(By.id("iframeButton")).click();

// — ALWAYS switch back to main page when done —————
driver.switchTo().defaultContent();

// — Method 2: Switch by name or id attribute —————
// HTML: <iframe name="paymentFrame" id="payFrame">
driver.switchTo().frame("paymentFrame"); // by name
// OR
driver.switchTo().frame("payFrame");      // by id

// Interact with content inside frame
driver.findElement(By.id("cardNumber")).sendKeys("4111111111111111");

driver.switchTo().defaultContent(); // switch back

// — Method 3: Switch by WebElement (MOST RELIABLE) —————
// This works even when name/id are absent or dynamic
WebElement frame = driver.findElement(By.cssSelector("iframe.content-frame"));
driver.switchTo().frame(frame);

driver.findElement(By.id("submitBtn")).click();

driver.switchTo().defaultContent();

// — Nested iFrames —————
// iFrame inside an iFrame – switch one level at a time
driver.switchTo().frame("outerFrame");
driver.switchTo().frame("innerFrame"); // now inside inner frame

driver.findElement(By.id("deepElement")).click();

driver.switchTo().parentFrame(); // back to outerFrame
driver.switchTo().defaultContent(); // back to main page
```

3.6 Handling Multiple Windows and Tabs

Clicking a link may open a new browser tab or window. Selenium does not automatically switch to it. You must switch manually using window handles. A window handle is a unique string ID for each open tab/window.

Example: Open a new window, work in it, then come back

```
// Step 1: Save the current (main) window handle BEFORE clicking
```



```
String mainWindowHandle = driver.getWindowHandle();
System.out.println("Main window: " + mainWindowHandle);

// Step 2: Click something that opens a new window/tab
driver.findElement(By.id("openNewWindowBtn")).click();

// Step 3: Get ALL window handles (includes main + new ones)
Set<String> allHandles = driver.getWindowHandles();
System.out.println("Total windows open: " + allHandles.size());

// Step 4: Switch to the new window
for (String handle : allHandles) {
    if (!handle.equals(mainWindowHandle)) {
        driver.switchTo().window(handle);
        break;
    }
}

// Step 5: Work in the new window
System.out.println("New window title: " + driver.getTitle());
System.out.println("New window URL: " + driver.getCurrentUrl());
driver.findElement(By.id("acceptButton")).click();

// Step 6: Close the new window
driver.close(); // closes ONLY the current window

// Step 7: Switch back to main window
driver.switchTo().window(mainWindowHandle);
System.out.println("Back to main: " + driver.getTitle());
```

Example: Open a new tab using JavaScript and work in it

```
String mainHandle = driver.getWindowHandle();

// Open a new tab using JavaScript
((JavascriptExecutor) driver).executeScript("window.open('https://google.com', '_blank');");

// Wait a moment for the tab to open
Thread.sleep(1000); // only for demo; use WebDriverWait in real tests

// Switch to the newly opened tab
for (String handle : driver.getWindowHandles()) {
    if (!handle.equals(mainHandle)) {
        driver.switchTo().window(handle);
    }
}

System.out.println("New tab: " + driver.getTitle());

// Close tab and return to main
driver.close();
driver.switchTo().window(mainHandle);
```

MODULE 4 Waits & Synchronization — The Most Critical Skill

Module 4: Waits & Synchronization

This is the single most important topic for writing STABLE Selenium tests. Most test failures are not caused by wrong locators — they are caused by timing issues. Modern web apps use JavaScript to load content dynamically. If your code runs before the element appears, you get a `NoSuchElementException` even though the element WILL appear in 0.5 seconds.

4.1 Why Timing Issues Happen

Here is a typical scenario that causes flaky tests:

```
driver.findElement(By.id("loginBtn")).click(); // click Login
// Page is now loading the dashboard...
// Dashboard might take 2 seconds to render...
driver.findElement(By.id("welcome-msg")).getText(); // BOOM — NoSuchElementException!
// Selenium looked for welcome-msg BEFORE it appeared on the page
```

The solution is **WAITS** — telling Selenium to be patient and keep checking until the element is ready.

WARNING: NEVER use `Thread.sleep(3000)` as a wait strategy. It wastes 3 full seconds even if the element loads in 0.1 seconds. And it still fails if the server is slow and takes 4 seconds. Proper waits are smarter.

4.2 Implicit Wait — The Simple Global Wait

Implicit wait tells WebDriver: 'If `findElement()` doesn't find the element immediately, keep polling for up to N seconds before throwing an exception.' You set it **ONCE** and it applies to every `findElement()` call.

Example: Setting implicit wait for the entire browser session

```
import java.time.Duration;

WebDriverManager.chromedriver().setup();
WebDriver driver = new ChromeDriver();

// Set implicit wait — applies to ALL findElement() calls from now on
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

driver.get("https://example.com");

// If this element takes 3 seconds to appear, Selenium will wait
// up to 10 seconds before throwing NoSuchElementException
WebElement element = driver.findElement(By.id("dynamicContent"));
System.out.println(element.getText());

// You can also set page load timeout:
driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30));
```

```
// And script execution timeout:
driver.manage().timeouts().scriptTimeout(Duration.ofSeconds(30));
```

NOTE: *Implicit wait has limitations. It only waits for the element to EXIST in DOM, not for it to be VISIBLE or CLICKABLE. Use Explicit Wait for those conditions.*

4.3 Explicit Wait — The Professional Approach

Explicit Wait is the most powerful and recommended wait strategy. You define EXACTLY what condition to wait for, and how long to wait. It checks the condition repeatedly (every 500ms by default) until it becomes true or timeout is reached.

Example: All the important ExpectedConditions you will use

```
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;
import java.time.Duration;

// Create a wait object - 15 second timeout
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));

// — 1. Wait for element to be VISIBLE (exists AND displayed) —————
// Use this most often - ensures element is rendered on screen
WebElement result = wait.until(
    ExpectedConditions.visibilityOfElementLocated(By.id("result"))
);
System.out.println(result.getText());

// — 2. Wait for element to be CLICKABLE (visible + enabled) —————
// Always use this before clicking a button
WebElement btn = wait.until(
    ExpectedConditions.elementToBeClickable(By.id("submitBtn"))
);
btn.click();

// — 3. Wait for element to be PRESENT in DOM (may not be visible) ———
// Element exists in HTML but might be hidden
WebElement hiddenEl = wait.until(
    ExpectedConditions.presenceOfElementLocated(By.id("hiddenField"))
);

// — 4. Wait for element to DISAPPEAR —————
// Useful for waiting for a loading spinner to go away
wait.until(
    ExpectedConditions.invisibilityOfElementLocated(By.id("spinner"))
);
System.out.println("Loading complete!");

// — 5. Wait for specific TEXT to appear in an element —————
wait.until(
    ExpectedConditions.textToBePresentInElementLocated(
        By.id("statusMsg"), "Order Placed Successfully"
    )
);

// — 6. Wait for URL to change —————
// After login, wait for URL to contain '/dashboard'
wait.until(ExpectedConditions.urlContains("/dashboard"));
System.out.println("Redirected to: " + driver.getCurrentUrl());
```

```
// — 7. Wait for page title —————
wait.until(ExpectedConditions.titleContains("Home"));
wait.until(ExpectedConditions.titleIs("Dashboard - MyApp"));

// — 8. Wait for Alert to appear —————
wait.until(ExpectedConditions.alertIsPresent());
driver.switchTo().alert().accept();

// — 9. Wait for number of windows/tabs —————
wait.until(ExpectedConditions.numberOfWindowsToBe(2));

// — 10. Wait for element to become SELECTED —————
wait.until(ExpectedConditions.elementToBeSelected(By.id("checkbox")));

// — 11. Wait for ALL elements matching a locator to be visible —————
List<WebElement> rows = wait.until(
    ExpectedConditions.visibilityOfAllElementsLocatedBy(By.tagName("tr"))
);

// — 12. Wait for element's ATTRIBUTE value —————
wait.until(
    ExpectedConditions.attributeContains(By.id("status"), "class", "success")
);
```

Example: Real-world login test using Explicit Wait

```
public void loginTest() {
    WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));

    driver.get("https://example.com/login");

    // Wait for login form to be visible before typing
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("loginForm")));

    driver.findElement(By.id("username")).sendKeys("admin");
    driver.findElement(By.id("password")).sendKeys("password123");

    // Wait for button to be clickable (not just visible)
    WebElement loginBtn = wait.until(
        ExpectedConditions.elementToBeClickable(By.id("loginBtn"))
    );
    loginBtn.click();

    // Wait for the loading spinner to disappear
    wait.until(
        ExpectedConditions.invisibilityOfElementLocated(By.id("loading"))
    );

    // Wait for successful navigation to dashboard
    wait.until(ExpectedConditions.urlContains("/dashboard"));

    // Now safely read the welcome message
    String welcomeMsg = wait.until(
        ExpectedConditions.visibilityOfElementLocated(By.id("welcomeMsg"))
    ).getText();

    System.out.println("Logged in! Welcome: " + welcomeMsg);
}
```

4.4 Fluent Wait — Custom Polling with Exception Ignoring

FluentWait is an extension of Explicit Wait that gives you full control: how often to check (polling interval) and which exceptions to ignore while waiting.

Example: FluentWait configuration and usage

```
import org.openqa.selenium.support.ui.FluentWait;
import org.openqa.selenium.NoSuchElementException;
import java.util.function.Function;

// Build a custom FluentWait
FluentWait<WebDriver> fluentWait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))    // max wait time: 30 seconds
    .pollingEvery(Duration.ofMillis(500))   // check every 500ms
    .ignoring(NoSuchElementException.class) // ignore this during polling
    .ignoring(StaleElementReferenceException.class); // and this

// Use it to find an element that loads slowly
WebElement element = fluentWait.until(driver -> {
    return driver.findElement(By.id("lateLoadingElement"));
});

// Wait for a CUSTOM condition (not just presence)
// Wait until a text field is non-empty
String result = fluentWait.until(driver -> {
    String text = driver.findElement(By.id("outputField")).getAttribute("value");
    return text.isEmpty() ? null : text; // return null to keep waiting
});
System.out.println("Field value: " + result);
```

4.5 Comparison — When to Use Which Wait

Wait Type	When to Use	Typical Usage
Implicit Wait	Simple projects, basic scripts	Set once at driver setup
Explicit Wait	Real frameworks, dynamic pages (RECOMMENDED)	Before every click/read on dynamic elements
Fluent Wait	Custom polling, ignoring specific exceptions	Complex async behaviors, retrying
Thread.sleep()	NEVER in real tests	Only for quick demos if absolutely necessary

3.7 Browser Cookies — Complete Guide

Cookies are small pieces of data stored in the browser. Selenium lets you read, add, modify, and delete cookies. This is useful for bypassing login (inject a session cookie), testing cookie consent banners, and verifying authentication state.

Example: Every cookie operation you will need

```
import org.openqa.selenium.Cookie;
import java.util.Set;

// — READ COOKIES —————

// Get ALL cookies on the current page
Set<Cookie> allCookies = driver.manage().getCookies();
System.out.println("Total cookies: " + allCookies.size());

// Print each cookie's details
for (Cookie cookie : allCookies) {
    System.out.println("Name: " + cookie.getName());
    System.out.println("Value: " + cookie.getValue());
    System.out.println("Domain: " + cookie.getDomain());
    System.out.println("Path: " + cookie.getPath());
    System.out.println("Expiry: " + cookie.getExpiry());
    System.out.println("Secure: " + cookie.isSecure());
    System.out.println("---");
}

// Get a SPECIFIC cookie by name
Cookie sessionCookie = driver.manage().getCookieNamed("session_id");
if (sessionCookie != null) {
    System.out.println("Session ID: " + sessionCookie.getValue());
}

// — ADD COOKIES (inject before navigating) —————
// You must be on the same domain before adding cookies
driver.get("https://example.com");

// Simple cookie
driver.manage().addCookie(new Cookie("username", "john_doe"));

// Cookie with domain, path, and expiry
Cookie fullCookie = new Cookie.Builder("auth_token", "eyJhbGciOiJIUzI1")
    .domain("example.com")
    .path("/")
    .isSecure(true)
    .build();
driver.manage().addCookie(fullCookie);

// Refresh page so the cookie takes effect
driver.navigate().refresh();

// — DELETE COOKIES —————
// Delete a specific cookie
driver.manage().deleteCookieNamed("username");

// Delete a specific cookie object
driver.manage().deleteCookie(sessionCookie);

// Delete ALL cookies
driver.manage().deleteAllCookies();
```

TIP: Injecting a session cookie is a powerful technique to skip the login UI in tests. Log in once manually, capture the session cookie value, then inject it in @BeforeMethod to bypass the login page entirely — saving seconds on every test.

3.8 Browser Window & Tab Management

Example: Complete window size and position control

```
import org.openqa.selenium.Dimension;
import org.openqa.selenium.Point;

// — Window size —————
driver.manage().window().maximize();           // Maximize to full screen
driver.manage().window().minimize();           // Minimize to taskbar
driver.manage().window().fullscreen();         // True fullscreen (F11 mode)

// Set a specific size (useful for responsive design testing)
driver.manage().window().setSize(new Dimension(1280, 800)); // desktop
driver.manage().window().setSize(new Dimension(768, 1024)); // tablet
driver.manage().window().setSize(new Dimension(375, 667));  // mobile

// Get current window size
Dimension size = driver.manage().window().getSize();
System.out.println("Width: " + size.getWidth() + " Height: " + size.getHeight());

// — Window position —————
driver.manage().window().setPosition(new Point(0, 0));      // Top-left corner
driver.manage().window().setPosition(new Point(100, 50));    // 100px right, 50px down

Point position = driver.manage().window().getPosition();
System.out.println("X: " + position.getX() + " Y: " + position.getY());
```

MODULE 5 Actions Class — Mouse, Keyboard, Drag & Drop

Module 5: Actions Class & Advanced Interactions

The Actions class is used for complex user gestures that cannot be done with just `element.click()` or `element.sendKeys()`. This includes hovering, right-clicking, double-clicking, drag and drop, keyboard shortcuts, and chaining multiple actions.

5.1 Setting Up the Actions Class

```
import org.openqa.selenium.interactions.Actions;

// Create an Actions object - pass your driver to it
Actions actions = new Actions(driver);

// IMPORTANT: Always call .perform() at the end to EXECUTE the action
// Without perform(), nothing happens!
```

5.2 Mouse Actions

Example: Hover — `moveToElement`

```
// — HOVER: Move mouse over an element (triggers CSS :hover states) —
// Very common for navigation menus that show submenus on hover

WebElement mainMenuProduct = driver.findElement(By.id("productsMenu"));

// Hover over the main menu
actions.moveToElement(mainMenuProduct).perform();

// Submenu is now visible - find and click a submenu item
WebElement subMenuItem = driver.findElement(By.id("laptopsSubmenu"));
subMenuItem.click();

// — CHAIN: Hover then click —————
actions.moveToElement(mainMenuProduct)
    .click(subMenuItem)
    .perform();

// — Move to element with offset (pixels from element's center) —————
actions.moveToElement(element, 10, 20).perform(); // 10px right, 20px down
```

Example: Click variations

```
WebElement element = driver.findElement(By.id("myElement"));

// — Double Click —————
// Used for: selecting text, activating edit mode in tables
actions.doubleClick(element).perform();
```



```
// — Right Click (Context Click) —————
// Opens right-click context menu
actions.contextClick(element).perform();

// After right-click, you can click a context menu item:
driver.findElement(By.xpath("//li[text()='Copy']")).click();

// — Click and Hold —————
actions.clickAndHold(element).perform();
Thread.sleep(2000); // hold for 2 seconds
actions.release().perform();

// — Move to an element and click —————
actions.moveToElement(element).click().perform();
```

Example: Drag and Drop — three approaches

```
WebElement source = driver.findElement(By.id("draggableItem"));
WebElement target = driver.findElement(By.id("dropZone"));

// — Method 1: dragAndDrop (simplest) —————
actions.dragAndDrop(source, target).perform();

// — Method 2: clickAndHold → moveToElement → release —————
// More reliable for some HTML5 drag-and-drop implementations
actions.clickAndHold(source)
    .moveToElement(target)
    .release(target)
    .perform();

// — Method 3: dragAndDropBy (move by pixel offset) —————
// Drag 300 pixels to the right, 0 pixels vertically
actions.dragAndDropBy(source, 300, 0).perform();

// Drag 0 right, 200 pixels down
actions.dragAndDropBy(source, 0, 200).perform();
```

5.3 Keyboard Actions

Example: Special keys and keyboard shortcuts

```
import org.openqa.selenium.Keys;

WebElement searchBox = driver.findElement(By.name("q"));

// — Send special keys directly to an element —————
searchBox.sendKeys("Selenium WebDriver");
searchBox.sendKeys(Keys.ENTER); // Press Enter
searchBox.sendKeys(Keys.TAB); // Move focus to next field
searchBox.sendKeys(Keys.ESCAPE); // Press Escape
searchBox.sendKeys(Keys.BACK_SPACE); // Delete one character
searchBox.sendKeys(Keys.DELETE); // Delete key
searchBox.sendKeys(Keys.HOME); // Go to start of line
searchBox.sendKeys(Keys.END); // Go to end of line
searchBox.sendKeys(Keys.ARROW_DOWN); // Arrow key down
searchBox.sendKeys(Keys.PAGE_DOWN); // Page Down

// — Keyboard Shortcuts using Keys.chord() —————
// chord() presses multiple keys simultaneously
```

```
// Select all text (Ctrl + A)
searchBox.sendKeys(Keys.chord(Keys.CONTROL, "a"));

// Copy (Ctrl + C)
searchBox.sendKeys(Keys.chord(Keys.CONTROL, "c"));

// Paste (Ctrl + V)
searchBox.sendKeys(Keys.chord(Keys.CONTROL, "v"));

// Undo (Ctrl + Z)
searchBox.sendKeys(Keys.chord(Keys.CONTROL, "z"));

// Select all, then delete
searchBox.sendKeys(Keys.chord(Keys.CONTROL, "a"));
searchBox.sendKeys(Keys.DELETE);

// — Keyboard actions using Actions class —————
// Press Shift while typing = types in UPPERCASE
actions.keyDown(Keys.SHIFT)
        .sendKeys("hello world")
        .keyUp(Keys.SHIFT)
        .perform();
// Result: types 'HELLO WORLD'

// Press Ctrl and click (multi-select in a list)
WebElement item1 = driver.findElement(By.id("item1"));
WebElement item3 = driver.findElement(By.id("item3"));
actions.click(item1)
        .keyDown(Keys.CONTROL)
        .click(item3)
        .keyUp(Keys.CONTROL)
        .perform();
```

5.4 Scrolling

Example: All scrolling techniques

```
import org.openqa.selenium.JavascriptExecutor;

JavascriptExecutor js = (JavascriptExecutor) driver;

// — Scroll to the BOTTOM of the page —————
js.executeScript("window.scrollTo(0, document.body.scrollHeight);");

// — Scroll to the TOP of the page —————
js.executeScript("window.scrollTo(0, 0);");

// — Scroll DOWN by a specific pixel amount —————
js.executeScript("window.scrollBy(0, 500);"); // down 500 pixels

// — Scroll UP by a specific pixel amount —————
js.executeScript("window.scrollBy(0, -300);"); // up 300 pixels

// — Scroll an element INTO VIEW —————
// Use this when an element is below/above the viewport
WebElement footer = driver.findElement(By.id("footer"));
js.executeScript("arguments[0].scrollIntoView(true);", footer);
// true = align to top of viewport
// false = align to bottom of viewport

// — Scroll to specific X,Y coordinates —————
```

```
js.executeScript("window.scrollTo(0, 1200);"); // scroll to 1200px from top

// — Selenium 4: Native scroll using Actions —————
// Scroll directly to an element (Selenium 4.2+)
actions.scrollToElement(footer).perform();

// Scroll by a specific amount from current position
actions.scrollByAmount(0, 400).perform(); // scroll 400px down
```

MODULE 6 JavaScript Executor & Screenshots

Module 6: JavaScript Executor & Screenshots

The JavaScript Executor lets you run any JavaScript code directly in the browser from your Java test. This is your escape hatch when Selenium's normal methods cannot do something — like clicking hidden elements, reading hidden values, or manipulating the DOM directly.

6.1 JavascriptExecutor — Every Use Case

Example: All common JavaScript Executor operations

```
import org.openqa.selenium.JavascriptExecutor;

// Cast the driver - ChromeDriver implements JavascriptExecutor
JavascriptExecutor js = (JavascriptExecutor) driver;

// — CLICK via JavaScript —————
// Use when normal .click() fails (element covered by overlay, etc.)
WebElement btn = driver.findElement(By.id("hiddenBtn"));
js.executeScript("arguments[0].click();", btn);

// — SET VALUE directly (bypass read-only or disabled inputs) —————
WebElement input = driver.findElement(By.id("readOnlyField"));
js.executeScript("arguments[0].value='New Value';", input);

// — GET value from element —————
String value = (String) js.executeScript(
    "return arguments[0].value;", input);
System.out.println("Field value: " + value);

// — HIGHLIGHT element (very useful for debugging) —————
js.executeScript(
    "arguments[0].style.border='3px solid red'; " +
    "arguments[0].style.backgroundColor='yellow';",
    btn);

// — READ page information —————
Long scrollY = (Long) js.executeScript("return window.scrollY;");
System.out.println("Current scroll position: " + scrollY);

Long pageHeight = (Long) js.executeScript("return document.body.scrollHeight;");
System.out.println("Full page height: " + pageHeight + "px");

// — NAVIGATE —————
js.executeScript("window.location.href='https://google.com';");

// — OPEN NEW TAB —————
js.executeScript("window.open('https://example.com', '_blank');");

// — REFRESH PAGE —————
js.executeScript("location.reload();");

// — RETURN a value from page —————
```

```
String title = (String) js.executeScript("return document.title;");
System.out.println("Title via JS: " + title);

// — Check if element is in viewport —————
Boolean inViewport = (Boolean) js.executeScript(
    "var rect = arguments[0].getBoundingClientRect();" +
    "return (rect.top >= 0 && rect.bottom <= window.innerHeight);",
    btn);
System.out.println("Is button visible in viewport: " + inViewport);
```

6.2 Taking Screenshots

Example: Full page and element-level screenshots

```
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.OutputType;
import org.apache.commons.io.FileUtils;
import java.io.File;

// — Full page screenshot —————
TakesScreenshot ts = (TakesScreenshot) driver;

// Get as FILE and save it
File screenshot = ts.getScreenshotAs(OutputType.FILE);
File destination = new File("screenshots/homepage.png");
FileUtils.copyFile(screenshot, destination);
System.out.println("Screenshot saved: " + destination.getAbsolutePath());

// — Screenshot as Base64 string (for embedding in HTML reports) —————
String base64 = ts.getScreenshotAs(OutputType.BASE64);
System.out.println("Base64 length: " + base64.length());

// — Screenshot as byte array —————
byte[] bytes = ts.getScreenshotAs(OutputType.BYTES);

// — Element-level screenshot (Selenium 4+) —————
// Captures ONLY the element, not the whole page
WebElement chart = driver.findElement(By.id("salesChart"));
File elementShot = chart.getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(elementShot, new File("screenshots/chart.png"));

// — Take screenshot with TIMESTAMP in filename —————
String timestamp = new java.text.SimpleDateFormat("yyyyMMdd_HH:mm:ss")
    .format(new java.util.Date());
File timed = new File("screenshots/shot_" + timestamp + ".png");
FileUtils.copyFile(ts.getScreenshotAs(OutputType.FILE), timed);
```

TIP: Best practice: Take a screenshot automatically on EVERY test failure. Add this to your @AfterMethod with ITestResult to check if the test failed.

MODULE 7 TestNG Framework — Annotations, Assertions, DataProvider

Module 7: TestNG Framework

TestNG (Test Next Generation) is a powerful testing framework that organizes your Selenium code into proper tests. Without TestNG, you'd have to manually run each script. TestNG gives you annotations for setup/teardown, assertion methods to verify results, grouping, parallel execution, and HTML reports.

7.1 TestNG Annotations — The Execution Order

Annotations are markers on methods that tell TestNG WHEN to run them. The execution order is:

@BeforeSuite	→ runs ONCE before all tests in the entire test suite
@BeforeClass	→ runs ONCE before all @Test methods in this class
@BeforeMethod	→ runs BEFORE EACH @Test method ← put browser launch here
@Test	→ the actual test method
@AfterMethod	→ runs AFTER EACH @Test method ← put browser quit here
@AfterClass	→ runs ONCE after all @Test methods in this class
@AfterSuite	→ runs ONCE after all tests in the entire test suite

Example: Complete TestNG test class with all annotations explained

```
import org.testng.annotations.*;
import org.testng.Assert;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import io.github.bonigarcia.wdm.WebDriverManager;
import java.time.Duration;

public class LoginTest {

    WebDriver driver;
    String baseUrl = "https://example.com";

    @BeforeSuite
    public void globalSetup() {
        System.out.println("=== Starting Test Suite ===");
        // Runs once. Good for: DB setup, test data preparation
    }

    @BeforeClass
    public void classSetup() {
        System.out.println("Setting up LoginTest class");
        // Runs once per class. Good for: reading config files
    }

    @BeforeMethod
    public void setUp() {
        // Runs before EVERY test — so each test gets a fresh browser
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.get(baseUrl + "/login");
    }
}
```

```

        System.out.println("Browser launched, navigated to login page");
    }

    @Test(description = "Verify login with valid credentials")
    public void testValidLogin() {
        driver.findElement(By.id("username")).sendKeys("admin");
        driver.findElement(By.id("password")).sendKeys("password123");
        driver.findElement(By.id("loginBtn")).click();

        // Verify we are now on the dashboard
        Assert.assertEquals(driver.getTitle(), "Dashboard",
            "Page title should be Dashboard after login");
        System.out.println("PASS: Valid login test passed");
    }

    @Test(description = "Verify error for invalid credentials")
    public void testInvalidLogin() {
        driver.findElement(By.id("username")).sendKeys("wronguser");
        driver.findElement(By.id("password")).sendKeys("wrongpass");
        driver.findElement(By.id("loginBtn")).click();

        // Verify error message is shown
        WebElement errorMsg = driver.findElement(By.id("errorMessage"));
        Assert.assertTrue(errorMsg.isDisplayed(), "Error message should be visible");
        Assert.assertEquals(errorMsg.getText(), "Invalid username or password");
    }

    @Test(description = "Verify login with empty fields")
    public void testEmptyLogin() {
        // Click login without entering credentials
        driver.findElement(By.id("loginBtn")).click();

        // Verify validation messages
        Assert.assertTrue(
            driver.findElement(By.id("usernameError")).isDisplayed(),
            "Username validation error should show"
        );
    }

    @AfterMethod
    public void tearDown() {
        // Runs after EVERY test - closes browser cleanly
        // Using null check in case setUp() failed
        if (driver != null) {
            driver.quit();
            System.out.println("Browser closed");
        }
    }

    @AfterClass
    public void classTeardown() {
        System.out.println("LoginTest class complete");
    }

    @AfterSuite
    public void globalTeardown() {
        System.out.println("=== Test Suite Complete ===");
    }
}

```

7.2 @Test Annotation Properties

Property	What It Does
description	Human-readable description of what the test does.
priority	Run order — lower number runs first (default: 0).
enabled = false	Skips this test. Useful for temporarily disabling a test.
groups = {"smoke"}	Tags the test into a group for selective execution.
dependsOnMethods = {"testA"}	This test only runs if testA passes.
expectedExceptions = {Exception.class}	Test PASSES if this exception is thrown.
timeOut = 5000	Test fails if it takes more than 5000ms (5 seconds).
invocationCount = 3	Runs this test 3 times (load/stability testing).
dataProvider = "myData"	Uses a DataProvider method to supply test data.

Example: Test annotation properties in action

```
// — Priority - controls execution order —————
@Test(priority = 1, description = "Verify page title")
public void verifyPageTitle() {
    Assert.assertEquals(driver.getTitle(), "Login Page");
}

@Test(priority = 2, description = "Verify login")
public void verifyLogin() {
    // Runs after priority 1
}

// — Disabled test —————
@Test(enabled = false, description = "Bug #123 - skip until fixed")
public void brokenTest() {
    // This test is skipped entirely
}

// — Groups - for selective test execution —————
@Test(groups = {"smoke", "regression"})
public void smokeTest() { ... }

@Test(groups = {"regression"})
public void detailedTest() { ... }

// — Expected exception - test PASSES when exception is thrown —————
@Test(expectedExceptions = { ArithmeticException.class })
public void testDivideByZero() {
    int result = 10 / 0; // Test passes because ArithmeticException is expected
}

// — Timeout - test fails if slower than N milliseconds —————
@Test(timeOut = 3000)
public void fastTest() {
    driver.get("https://example.com");
    // Must complete within 3000ms or test fails with TimeoutException
}
```


7.3 Assertions — Hard vs Soft

Assertions are how you VERIFY that your application behaves correctly. A failed assertion means the test fails. Hard assertions stop the test immediately. Soft assertions collect all failures and report them all at the end.

Example: All Assert methods with real examples

```
import org.testng.Assert;

// — assertEquals: actual must EQUAL expected —————
// Fails if actual != expected
String actualTitle = driver.getTitle();
Assert.assertEquals(actualTitle, "Dashboard", "Title should be Dashboard");

// — assertNotEquals: actual must NOT equal expected —————
Assert.assertNotEquals(driver.getCurrentUrl(), "https://example.com/login",
    "Should NOT be on login page after logging in");

// — assertTrue: condition must be TRUE —————
WebElement welcomeMsg = driver.findElement(By.id("welcome"));
Assert.assertTrue(welcomeMsg.isDisplayed(), "Welcome message should be visible");

// — assertFalse: condition must be FALSE —————
WebElement errorMsg = driver.findElement(By.id("error"));
Assert.assertFalse(errorMsg.isDisplayed(), "Error should NOT be visible");

// — assertNull / assertNotNull —————
String loggedInUser = getUserFromSession();
Assert.assertNotNull(loggedInUser, "User should be logged in (not null)");

// — assertEqualsIgnoreCase (using Java) —————
// TestNG doesn't have this - use assertTrue with equalsIgnoreCase
Assert.assertTrue(
    driver.getTitle().equalsIgnoreCase("dashboard"),
    "Title check is case-insensitive"
);

// — Verify text CONTAINS a substring —————
String successMsg = driver.findElement(By.id("successMsg")).getText();
Assert.assertTrue(
    successMsg.contains("successfully"),
    "Success message should contain 'successfully', but was: " + successMsg
);
```

Example: SoftAssert — collect all failures, report at end

```
import org.testng.asserts.SoftAssert;

// Scenario: Check multiple things on the dashboard page after login

SoftAssert soft = new SoftAssert();

// Check page title
soft.assertEquals(driver.getTitle(), "Dashboard", "Title mismatch");

// Check welcome message is displayed
soft.assertTrue(
    driver.findElement(By.id("welcomeMsg")).isDisplayed(),
    "Welcome message not shown"
```

```

);

// Check username is shown correctly
String displayedUser = driver.findElement(By.id("loggedInUser")).getText();
soft.assertEquals(displayedUser, "Admin", "Username should be Admin");

// Check navigation menu is present
soft.assertTrue(
    driver.findElement(By.id("navMenu")).isDisplayed(),
    "Navigation menu should be visible"
);

// — CRITICAL: Must call assertAll() at the end —————
// Without this, failures are SILENTLY IGNORED
// assertAll() throws AssertionError listing ALL failures at once
soft.assertAll();

// If 2 of the 4 checks failed, you'll see BOTH failure messages,
// not just the first one (unlike hard assertions).

```

7.4 Data-Driven Testing with @DataProvider

Instead of writing separate test methods for each set of test data, use `@DataProvider` to run the SAME test with multiple data sets automatically. This is essential for testing login with valid/invalid/edge case credentials, form validation, etc.

Example: Data-driven login test with multiple user types

```

import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class LoginDataDrivenTest {

    // — @DataProvider: method that returns test data —————
    // Returns Object[][] — each inner array is one test run's data
    @DataProvider(name = "loginCredentials")
    public Object[][] loginData() {
        return new Object[][] {
            // { username, password, expectedResult }
            { "admin", "password123", "Dashboard" }, // valid admin
            { "user@test.com", "userpass", "Dashboard" }, // valid user
            { "wronguser", "wrongpass", "error" }, // invalid creds
            { "", "", "error" }, // empty fields
            { "admin", "", "error" }, // missing password
            { " ", "password123", "error" }, // whitespace user
        };
    }

    // — @Test using the DataProvider —————
    // TestNG runs this method 6 times — once per row of data
    @Test(dataProvider = "loginCredentials")
    public void testLogin(String username, String password, String expected) {
        driver.get("https://example.com/login");

        driver.findElement(By.id("username")).sendKeys(username);
        driver.findElement(By.id("password")).sendKeys(password);
        driver.findElement(By.id("loginBtn")).click();

        if (expected.equals("Dashboard")) {
            // Expect successful login

```

```
        Assert.assertEquals(driver.getTitle(), "Dashboard",
            "Login with '" + username + "' should succeed");
    } else {
        // Expect error message
        Assert.assertTrue(
            driver.findElement(By.id("errorMsg")).isDisplayed(),
            "Login with '" + username + "' should fail with error"
        );
    }
}

// TestNG output shows each run as a separate test:
// PASS: testLogin('admin', 'password123', 'Dashboard')
// PASS: testLogin('user@test.com', 'userpass', 'Dashboard')
// PASS: testLogin('wronguser', 'wrongpass', 'error')
// ... etc
```

MODULE 8 Page Object Model (POM) — Professional Framework

Module 8: Page Object Model (POM)

The Page Object Model is the most important design pattern in Selenium automation. Every professional framework uses it. The idea is simple: create a separate Java class for each page of your web application. Each page class contains the locators and action methods for that page. Your test classes only call these methods — they never deal with locators directly.

8.1 The Problem POM Solves

Without POM, imagine you have this button: `By.cssSelector(".btn.btn-login")`. This locator is used in 15 different test files. One day the button's CSS class changes to `.login-submit-button`. Now you must find and fix it in ALL 15 files — and you will miss some.

With POM, the locator lives in ONE place: `LoginPage.java`. You change it there, and all 15 tests automatically use the updated locator.

8.2 Complete Project Structure

```
src/
├── main/java/
│   ├── pages/
│   │   ├── BasePage.java      ← shared methods for all pages
│   │   ├── LoginPage.java     ← locators + actions for login page
│   │   ├── DashboardPage.java ← locators + actions for dashboard
│   │   ├── ProductPage.java   ← locators + actions for products
│   │   └── CartPage.java      ← locators + actions for shopping cart
│   └── utils/
│       ├── DriverFactory.java ← creates and manages WebDriver
│       └── ConfigReader.java  ← reads config.properties
├── test/java/
│   ├── base/
│   │   └── BaseTest.java      ← @BeforeMethod / @AfterMethod
│   └── tests/
│       ├── LoginTest.java     ← test methods only, no locators
│       └── ProductTest.java
├── src/test/resources/
│   ├── testng.xml            ← TestNG suite config
│   └── config.properties     ← URL, browser, credentials
```

8.3 BasePage.java — Shared Methods for All Pages

Every page extends `BasePage`. `BasePage` holds reusable methods like `click`, `getText`, `waitForVisible` — so you write them ONCE and use them everywhere.

```
package pages;
```

```
import org.openqa.selenium.*;
import org.openqa.selenium.support.ui.*;
import java.time.Duration;

public class BasePage {

    protected WebDriver driver;
    protected WebDriverWait wait;

    // Constructor: every page needs a driver
    public BasePage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(15));
    }

    // — Reusable methods all pages can use —————

    // Wait for element to be visible, then return it
    protected WebElement waitForVisible(By locator) {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    }

    // Wait for element to be clickable, then click it
    protected void click(By locator) {
        wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
    }

    // Clear field and type text
    protected void type(By locator, String text) {
        WebElement element = waitForVisible(locator);
        element.clear();
        element.sendKeys(text);
    }

    // Get text of an element
    protected String getText(By locator) {
        return waitForVisible(locator).getText();
    }

    // Check if element is displayed (safe – returns false if not found)
    protected boolean isDisplayed(By locator) {
        try {
            return driver.findElement(locator).isDisplayed();
        } catch (NoSuchElementException e) {
            return false;
        }
    }

    // Get attribute value
    protected String getAttribute(By locator, String attribute) {
        return waitForVisible(locator).getAttribute(attribute);
    }

    // Wait for page URL to contain a string
    protected void waitForUrl(String urlPart) {
        wait.until(ExpectedConditions.urlContains(urlPart));
    }

    // Wait for an element to disappear
    protected void waitForInvisible(By locator) {
        wait.until(ExpectedConditions.invisibilityOfElementLocated(locator));
    }
}
```

```
// Get the current page title
protected String getPageTitle() {
    return driver.getTitle();
}
}
```

8.4 LoginPage.java — A Complete Page Object

```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage extends BasePage {

    // — ALL locators for the login page in ONE place —————
    private final By usernameInput = By.id("username");
    private final By passwordInput = By.id("password");
    private final By loginButton = By.id("loginBtn");
    private final By errorMessage = By.cssSelector(".error-alert");
    private final By forgotPassLink = By.linkText("Forgot Password?");
    private final By rememberMe = By.id("rememberMe");
    private final By pageHeading = By.tagName("h1");

    // Constructor
    public LoginPage(WebDriver driver) {
        super(driver); // Call BasePage constructor
    }

    // — ACTION METHODS — what you can DO on this page —————

    // Type into username field
    public LoginPage enterUsername(String username) {
        type(usernameInput, username);
        return this; // Return 'this' for method chaining
    }

    // Type into password field
    public LoginPage enterPassword(String password) {
        type(passwordInput, password);
        return this;
    }

    // Click login button — returns DashboardPage (fluent return)
    public DashboardPage clickLoginButton() {
        click(loginButton);
        return new DashboardPage(driver);
    }

    // Convenience method: full login in one call
    public DashboardPage loginAs(String username, String password) {
        return enterUsername(username)
            .enterPassword(password)
            .clickLoginButton();
    }

    // Click login expecting failure (stays on login page)
    public LoginPage clickLoginExpectingFailure() {
        click(loginButton);
        return this;
    }
}
```

```
// Click Forgot Password link
public void clickForgotPassword() {
    click(forgotPassLink);
}

// Toggle Remember Me checkbox
public void checkRememberMe() {
    if (!driver.findElement(rememberMe).isSelected()) {
        click(rememberMe);
    }
}

// — GETTER METHODS — reading page content —————

public String getErrorMessage() {
    return getText(errorMessage);
}

public boolean isErrorDisplayed() {
    return isDisplayed(errorMessage);
}

public String getPageHeading() {
    return getText(pageHeading);
}
}
```

8.5 DashboardPage.java

```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class DashboardPage extends BasePage {

    private final By welcomeMessage = By.id("welcomeMsg");
    private final By loggedInUser = By.id("userDisplay");
    private final By navigationMenu = By.id("navMenu");
    private final By logoutButton = By.id("logoutBtn");
    private final By productsLink = By.linkText("Products");

    public DashboardPage(WebDriver driver) {
        super(driver);
        // Verify we're actually on the dashboard
        waitForUrl("/dashboard");
    }

    public String getWelcomeMessage() {
        return getText(welcomeMessage);
    }

    public String getLoggedInUsername() {
        return getText(loggedInUser);
    }

    public boolean isNavigationMenuVisible() {
        return isDisplayed(navigationMenu);
    }

    public LoginPage logout() {
```

```

        click(logoutButton);
        return new LoginPage(driver);
    }

    public ProductPage goToProducts() {
        click(productsLink);
        return new ProductPage(driver);
    }
}

```

8.6 BaseTest.java — Browser Lifecycle for All Tests

```

package base;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.testng.ITestResult;
import org.testng.annotations.*;
import io.github.bonigarcia.wdm.WebDriverManager;
import org.apache.commons.io.FileUtils;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.OutputType;
import java.io.File;
import java.time.Duration;

public class BaseTest {

    protected WebDriver driver;
    protected String baseUrl = "https://example.com";

    @BeforeMethod
    public void setUp() {
        WebDriverManager.chromedriver().setup();

        // Chrome options for better stability
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--start-maximized");
        options.addArguments("--disable-notifications");
        options.addArguments("--disable-popup-blocking");

        driver = new ChromeDriver(options);
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30));
        driver.get(baseUrl);
    }

    @AfterMethod
    public void tearDown(ITestResult result) {
        // If test FAILED, take a screenshot
        if (result.getStatus() == ITestResult.FAILURE) {
            takeScreenshot(result.getName());
        }

        // Always close browser
        if (driver != null) {
            driver.quit();
        }
    }

    private void takeScreenshot(String testName) {
        try {

```



```

        String timestamp = new java.text.SimpleDateFormat("yyyyMMdd_HHmss")
            .format(new java.util.Date());
        File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
        File dest = new File("screenshots/FAIL_" + testName + "_" + timestamp +
            ".png");
        FileUtils.copyFile(src, dest);
        System.out.println("Screenshot: " + dest.getAbsolutePath());
    } catch (Exception e) {
        System.err.println("Could not take screenshot: " + e.getMessage());
    }
}
}

```

8.7 LoginTest.java — The Clean Test Using POM

Notice how the test reads like plain English. There are NO locators, NO findElement calls — all of that is in the page classes.

```

package tests;

import base.BaseTest;
import pages.LoginPage;
import pages.DashboardPage;
import org.testng.Assert;
import org.testng.annotations.*;

public class LoginTest extends BaseTest {

    LoginPage loginPage;

    @BeforeMethod
    public void navigateToLogin() {
        driver.get(baseUrl + "/login");
        loginPage = new LoginPage(driver);
    }

    @Test(description = "Valid login navigates to dashboard")
    public void testValidLogin() {
        // Reads like a user story:
        DashboardPage dashboard = loginPage.loginAs("admin", "password123");

        Assert.assertEquals(dashboard.getPageTitle(), "Dashboard");
        Assert.assertTrue(dashboard.isNavigationMenuVisible());
        Assert.assertEquals(dashboard.getLoggedInUsername(), "Admin");
    }

    @Test(description = "Invalid login shows error message")
    public void testInvalidLogin() {
        LoginPage result = loginPage
            .enterUsername("invalid")
            .enterPassword("wrongpass")
            .clickLoginExpectingFailure();

        Assert.assertTrue(result.isErrorDisplayed());
        Assert.assertEquals(result.getErrorMessage(), "Invalid username or password");
    }

    @Test(description = "Logout returns to login page")
    public void testLogout() {
        DashboardPage dashboard = loginPage.loginAs("admin", "password123");
        LoginPage afterLogout = dashboard.logout();
    }
}

```

```
Assert.assertEquals(afterLogout.getPageHeading(), "Login");
Assert.assertEquals(driver.getCurrentUrl(), baseUrl + "/login");
    }
}
```

8.8 Page Factory with @FindBy — Alternative to By Objects

Page Factory is another way to define locators. Instead of By objects, you use @FindBy annotations directly on WebElement fields. PageFactory.initElements() then injects the elements automatically.

Example: LoginPage using Page Factory with @FindBy annotations

```
package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.FindAll;
import org.openqa.selenium.support.PageFactory;
import java.util.List;

public class LoginPagePF {

    // — @FindBy: locate by a single strategy —————

    @FindBy(id = "username")
    private WebElement usernameField;

    @FindBy(id = "password")
    private WebElement passwordField;

    @FindBy(css = ".btn-login")
    private WebElement loginButton;

    @FindBy(xpath = "//div[@class='error-alert']")
    private WebElement errorMessage;

    @FindBy(linkText = "Forgot Password?")
    private WebElement forgotPassLink;

    // — @FindBy with name —————
    @FindBy(name = "rememberMe")
    private WebElement rememberMeCheckbox;

    // — @FindAll: find elements matching ANY of the given @FindBy —————
    // Returns a List — useful for catching multiple possible locators
    @FindAll({
        @FindBy(id = "errorMsg"),
        @FindBy(css = ".alert-danger"),
        @FindBy(xpath = "//p[@class='error']")
    })
    private List<WebElement> allErrorElements;

    // Constructor MUST call initElements to wire up the annotations
    public LoginPagePF(WebDriver driver) {
        PageFactory.initElements(driver, this);
    }

    public void enterUsername(String username) {
        usernameField.clear();
        usernameField.sendKeys(username);
    }

    public void enterPassword(String password) {
        passwordField.clear();
        passwordField.sendKeys(password);
    }
}
```

```

    }

    public void clickLogin() {
        loginButton.click();
    }

    public void login(String user, String pass) {
        enterUsername(user);
        enterPassword(pass);
        clickLogin();
    }

    public boolean isErrorDisplayed() {
        return errorMessage.isDisplayed();
    }

    public String getErrorText() {
        return errorMessage.getText();
    }
}

```

WARNING: A key difference from *By* objects: *@FindBy* elements are looked up every time you access them — so *StaleElementReferenceException* is less common. However, *isDisplayed()* will still throw *NoSuchElementException* if the element doesn't exist at all.

8.9 Thread-Safe Parallel Execution with ThreadLocal

When running tests in parallel (multiple threads at the same time), each thread needs its OWN *WebDriver* instance. If two tests share one driver object, they will interfere with each other. *ThreadLocal* solves this by giving each thread its own private copy.

```

package utils;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import io.github.bonigarcia.wdm.WebDriverManager;

public class DriverManager {

    // ThreadLocal<WebDriver> = each thread gets its own WebDriver
    // Thread A gets its own driver, Thread B gets its own driver
    // They never share or interfere with each other
    private static ThreadLocal<WebDriver> driverThread = new ThreadLocal<>();

    // Get the driver for the CURRENT thread
    public static WebDriver getDriver() {
        return driverThread.get();
    }

    // Create a new driver for the CURRENT thread
    public static void initDriver(String browser) {
        WebDriverManager.chromedriver().setup();
        driverThread.set(new ChromeDriver());
        getDriver().manage().window().maximize();
    }

    // Quit and remove driver for the CURRENT thread
    public static void quitDriver() {
        if (getDriver() != null) {
            getDriver().quit();
        }
    }
}

```

```
        driverThread.remove(); // IMPORTANT: prevent memory leak
    }
}

// BaseTest using DriverManager:
public class BaseTest {

    @BeforeMethod
    public void setUp() {
        DriverManager.initDriver(ConfigReader.get("browser"));
        DriverManager.getDriver().get(ConfigReader.get("baseUrl"));
    }

    @AfterMethod
    public void tearDown() {
        DriverManager.quitDriver();
    }

    // Any test or page can call DriverManager.getDriver() safely
    // Each parallel thread will get its own isolated driver
}
```

MODULE 9 Configuration, Reporting & Data-Driven Testing

Module 9: Configuration, Reporting & Excel Data Provider

9.1 config.properties — Externalizing Test Data

Never hardcode URLs, usernames, or passwords directly in your test code. Store them in a properties file so you can change them without touching code.

```
# src/test/resources/config.properties

# Application URLs
baseUrl=https://example.com
apiUrl=https://api.example.com

# Browser settings
browser=chrome
headless=false
timeout=15

# Test credentials
adminUsername=admin
adminPassword=Password123!
testUserEmail=testuser@example.com
```

```
// ConfigReader.java
package utils;

import java.io.*;
import java.util.Properties;

public class ConfigReader {

    private static Properties properties;

    static {
        properties = new Properties();
        try (InputStream input = ConfigReader.class
            .getResourceAsStream("/config.properties")) {
            properties.load(input);
        } catch (IOException e) {
            throw new RuntimeException("Cannot load config.properties: " + e);
        }
    }

    public static String get(String key) {
        String value = properties.getProperty(key);
        if (value == null) throw new RuntimeException("Key not found: " + key);
        return value;
    }
}
```

```

    public static int getInt(String key) {
        return Integer.parseInt(get(key));
    }

    public static boolean getBoolean(String key) {
        return Boolean.parseBoolean(get(key));
    }
}

// Usage anywhere in your code:
String url      = ConfigReader.get("baseUrl");
String browser  = ConfigReader.get("browser");
int timeout     = ConfigReader.getInt("timeout");

```

9.2 testng.xml — Suite Configuration

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">

<suite name="Selenium Master Suite" parallel="methods" thread-count="3">

    <!-- Listeners for reporting -->
    <listeners>
        <listener class-name="listeners.ExtentReportListener"/>
    </listeners>

    <!-- Smoke Tests - quick sanity check -->
    <test name="Smoke Tests">
        <groups>
            <run><include name="smoke"/></run>
        </groups>
        <classes>
            <class name="tests.LoginTest"/>
            <class name="tests.HomePageTest"/>
        </classes>
    </test>

    <!-- Full Regression Suite -->
    <test name="Regression Tests">
        <groups>
            <run><include name="regression"/></run>
        </groups>
        <classes>
            <class name="tests.LoginTest"/>
            <class name="tests.ProductTest"/>
            <class name="tests.CartTest"/>
            <class name="tests.CheckoutTest"/>
        </classes>
    </test>

</suite>

```

9.3 Extent Reports — Beautiful HTML Test Reports

```

<!-- Add to pom.xml -->
<dependency>
    <groupId>com.aventstack</groupId>
    <artifactId>extentreports</artifactId>
    <version>5.1.1</version>
</dependency>

```

```
// ExtentReportListener.java
package listeners;

import com.aventstack.extentreports.*;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;
import com.aventstack.extentreports.reporter.configuration.Theme;
import org.testng.*;
import org.openqa.selenium.*;
import org.openqa.selenium.OutputType;

public class ExtentReportListener implements ITestListener {

    private static ExtentReports extent;
    private static ThreadLocal<ExtentTest> test = new ThreadLocal<>();

    @Override
    public void onStart(ISuite suite) {
        ExtentSparkReporter reporter = new
ExtentSparkReporter("reports/TestReport.html");
        reporter.config().setTheme(Theme.DARK);
        reporter.config().setDocumentTitle("Selenium Test Results");
        reporter.config().setReportName("Automation Suite Report");

        extent = new ExtentReports();
        extent.setSystemInfo("OS", System.getProperty("os.name"));
        extent.setSystemInfo("Browser", ConfigReader.get("browser"));
        extent.attachReporter(reporter);
    }

    @Override
    public void onTestStart(ITestResult result) {
        ExtentTest extentTest = extent.createTest(
            result.getMethod().getMethodName(),
            result.getMethod().getDescription()
        );
        test.set(extentTest);
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        test.get().pass("Test PASSED");
    }

    @Override
    public void onTestFailure(ITestResult result) {
        test.get().fail(result.getThrowable());

        // Attach screenshot to report on failure
        try {
            WebDriver driver = ((BaseTest) result.getInstance()).driver;
            String base64 = ((TakesScreenshot) driver)
                .getScreenshotAs(OutputType.BASE64);
            test.get().addScreenCaptureFromBase64String(base64, "Failure Screenshot");
        } catch (Exception e) {
            test.get().warning("Could not capture screenshot: " + e.getMessage());
        }
    }

    @Override
    public void onTestSkipped(ITestResult result) {
        test.get().skip(result.getThrowable());
    }
}
```



```
@Override
public void onFinish(ISuite suite) {
    extent.flush(); // MUST call flush to write HTML to disk
    System.out.println("Report generated: reports/TestReport.html");
}
}
```

Module 10: Advanced Topics

10.1 Cross-Browser Testing

Professional frameworks run the same tests on Chrome, Firefox, and Edge. The browser should be configurable without changing code.

```
// DriverFactory.java - creates browser based on config
package utils;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.*;
import org.openqa.selenium.firefox.*;
import org.openqa.selenium.edge.*;
import io.github.bonigarcia.wdm.WebDriverManager;

public class DriverFactory {

    public static WebDriver createDriver(String browser, boolean headless) {
        switch (browser.toLowerCase().trim()) {

            case "chrome":
                WebDriverManager.chromedriver().setup();
                ChromeOptions chromeOpts = new ChromeOptions();
                if (headless) {
                    chromeOpts.addArguments("--headless");
                    chromeOpts.addArguments("--window-size=1920,1080");
                }
                chromeOpts.addArguments("--disable-notifications");
                return new ChromeDriver(chromeOpts);

            case "firefox":
                WebDriverManager.firefoxdriver().setup();
                FirefoxOptions ffOpts = new FirefoxOptions();
                if (headless) ffOpts.addArguments("--headless");
                return new FirefoxDriver(ffOpts);

            case "edge":
                WebDriverManager.edgedriver().setup();
                EdgeOptions edgeOpts = new EdgeOptions();
                if (headless) edgeOpts.addArguments("--headless");
                return new EdgeDriver(edgeOpts);

            default:
                throw new IllegalArgumentException(
                    "Unknown browser: '" + browser + "'. Use chrome, firefox, or
edge.");
        }
    }
}

// Usage in BaseTest.java:
String browser = ConfigReader.get("browser");
```

```
boolean headless = ConfigReader.getBoolean("headless");
driver = DriverFactory.createDriver(browser, headless);
```

10.2 Selenium Grid — Parallel Testing Across Machines

Selenium Grid lets you run tests simultaneously across multiple browsers and machines, dramatically reducing total execution time.

```
// Terminal commands to start Selenium Grid:

// 1. Start the Hub (central coordinator) — run on your main server:
//   java -jar selenium-server-4.18.1.jar hub
//   Hub is now available at: http://localhost:4444

// 2. Start Node(s) (actual browser machines):
//   java -jar selenium-server-4.18.1.jar node --hub http://localhost:4444

// 3. View Grid console: open http://localhost:4444 in browser

// — Connect your tests to Selenium Grid —————
import org.openqa.selenium.remote.RemoteWebDriver;
import java.net.URL;

public static WebDriver createRemoteDriver(String browser) throws Exception {
    ChromeOptions options = new ChromeOptions();
    // OR: FirefoxOptions, EdgeOptions

    // Connect to Grid Hub
    return new RemoteWebDriver(
        new URL("http://localhost:4444"),
        options
    );
}

// Your test code doesn't change AT ALL — same findElement, same clicks
// The difference: execution happens on the Grid machine, not your laptop
```

10.3 Cucumber BDD — Plain English Tests

BDD (Behavior Driven Development) lets business analysts, QA engineers, and developers collaborate by writing tests in plain English using Gherkin syntax.

```
<!-- Add to pom.xml -->
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>7.15.0</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-testng</artifactId>
  <version>7.15.0</version>
  <scope>test</scope>
</dependency>
```

```
# src/test/resources/features/Login.feature
# Written in plain English — readable by everyone
```

Feature: User Authentication

As a user

I want to log in to the application

So that I can access my account

Scenario: Successful login with valid credentials

Given I navigate to the login page

When I enter username "admin" and password "password123"

And I click the Login button

Then I should see the Dashboard page

And the welcome message should contain "Welcome, Admin"

Scenario: Failed login with invalid credentials

Given I navigate to the login page

When I enter username "hacker" and password "wrong"

And I click the Login button

Then I should see an error message "Invalid username or password"

Scenario Outline: Login with multiple user types

Given I navigate to the login page

When I enter username "<username>" and password "<password>"

And I click the Login button

Then the result should be "<result>"

Examples:

username	password	result	
admin	password123	dashboard	
user1	userpass	dashboard	
wrong	bad	error	

```
// LoginSteps.java - maps Gherkin steps to Selenium actions
package stepdefs;
```

```
import io.cucumber.java.en.*;
import org.testng.Assert;
import pages.*;
```

```
public class LoginSteps {
```

```
    WebDriver driver = DriverFactory.createDriver("chrome", false);
    LoginPage loginPage;
```

```
    @Given("I navigate to the login page")
```

```
    public void navigateToLogin() {
        driver.get(ConfigReader.get("baseUrl") + "/login");
        loginPage = new LoginPage(driver);
    }
```

```
    @When("I enter username {string} and password {string}")
```

```
    public void enterCredentials(String username, String password) {
        loginPage.enterUsername(username);
        loginPage.enterPassword(password);
    }
```

```
    @And("I click the Login button")
```

```
    public void clickLogin() {
        loginPage.clickLoginButton();
    }
```

```
    @Then("I should see the Dashboard page")
```

```
    public void verifyDashboard() {
        Assert.assertEquals(driver.getTitle(), "Dashboard");
    }
```

```
}

@Then("I should see an error message {string}")
public void verifyError(String expectedMsg) {
    Assert.assertEquals(loginPage.getErrorMessage(), expectedMsg);
}
}
```

10.4 GitHub Actions CI/CD — Auto-Run Tests on Every Push

```
# .github/workflows/selenium-ci.yml
name: Selenium Test Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  selenium-tests:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Java 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Cache Maven dependencies
        uses: actions/cache@v3
        with:
          path: ~/.m2
          key: m2-${{ hashFiles('**/pom.xml') }}

      - name: Run Selenium tests (headless)
        run: mvn test -Dbrowser=chrome -Dheadless=true

      - name: Upload Extent Report
        if: always() # Upload even if tests fail
        uses: actions/upload-artifact@v3
        with:
          name: test-report
          path: reports/TestReport.html

      - name: Upload failure screenshots
        if: failure()
        uses: actions/upload-artifact@v3
        with:
          name: failure-screenshots
          path: screenshots/
```

MODULE 11 Best Practices, Debugging & Common Errors

Module 11: Best Practices, Debugging & Common Errors

11.1 The Golden Rules of Selenium Automation

19. ALWAYS use Explicit Wait for dynamic elements — never `Thread.sleep()`
20. ALWAYS call `driver.quit()` — use a finally block or `@AfterMethod` to guarantee it
21. ALWAYS use Page Object Model — no locators in test classes
22. ALWAYS take a screenshot on test failure for debugging
23. ALWAYS verify your element exists before interacting — use `safe isDisplayed()`
24. NEVER use absolute XPaths like `/html/body/div[1]` — they break on any DOM change
25. NEVER hardcode URLs or credentials — use `config.properties`
26. NEVER make tests depend on each other — every test must be independent
27. KEEP tests focused — one test should verify one thing
28. NAME tests descriptively — `testValidLoginNavigatesToDashboard` not `test1`

11.2 Common Exceptions — Causes & Fixes

Exception	Cause & How to Fix
<code>NoSuchElementException</code>	Element not found with your locator. Fix: verify locator in DevTools, add Explicit Wait, ensure page has loaded.
<code>StaleElementReferenceException</code>	Element was found, but the DOM re-rendered. The reference is now stale. Fix: find the element again inside the action, or use Fluent Wait.
<code>ElementClickInterceptedException</code>	Another element (overlay, popup, cookie banner) is covering your element. Fix: close the covering element first, scroll into view, or use JS click.
<code>TimeoutException</code>	Wait condition was not met within the timeout. Fix: increase timeout, verify the condition is reachable, check for page errors.
<code>InvalidSelectorException</code>	Your XPath or CSS has a syntax error. Fix: test the selector in Chrome DevTools Console first.
<code>WebDriverException</code> (session not created)	ChromeDriver version doesn't match Chrome. Fix: update WebDriverManager or specify a version.

ElementNotInteractableException	Element exists but is not interactable (hidden, off-screen, behind overlay). Fix: wait for it to be visible, scroll into view, use JS.
NoAlertPresentException	Tried to switch to an alert but none exists. Fix: wait for alertIsPresent() before switching.
NoSuchWindowException	Tried to switch to a window handle that was closed. Fix: verify the handle still exists before switching.

11.3 Debugging Techniques

Example: Debug helpers — slow down, highlight, pause

```
// — Highlight elements to see what Selenium is interacting with ———
public void highlightElement(WebElement element) {
    JavascriptExecutor js = (JavascriptExecutor) driver;
    js.executeScript(
        "arguments[0].style.border = '3px solid red';",
        element
    );
    Thread.sleep(500); // Pause so you can see it
    js.executeScript(
        "arguments[0].style.border = '';",
        element
    );
}

// — Print ALL attributes of an element —————
WebElement el = driver.findElement(By.id("myElement"));
System.out.println("Tag:      " + el.getTagName());
System.out.println("Text:     " + el.getText());
System.out.println("id attr:   " + el.getAttribute("id"));
System.out.println("class attr: " + el.getAttribute("class"));
System.out.println("value attr: " + el.getAttribute("value"));
System.out.println("Is visible: " + el.isDisplayed());
System.out.println("Is enabled: " + el.isEnabled());

// — Test your XPath in Chrome Console —————
// Press F12 → Console → type:
// $x("//button[@id='loginBtn']")      → XPath test
// $$("button#loginBtn")              → CSS Selector test

// — Print all links on the current page —————
List<WebElement> links = driver.findElements(By.tagName("a"));
for (WebElement link : links) {
    String href = link.getAttribute("href");
    String text = link.getText();
    if (href != null && !href.isEmpty()) {
        System.out.printf("Link: %-30s -> %s\n", text, href);
    }
}
```

11.4 StaleElementReferenceException — Deep Fix

Example: The right way to handle stale element references

```
// — WHY IT HAPPENS —————
// WebElement el = driver.findElement(By.id("btn")); // found at time T
```

```
// ... page updates (AJAX, React re-render, navigation) ...
// el.click(); // BOOM - StaleElementReferenceException
// // 'el' now points to a DOM node that no longer exists

// — FIX 1: Re-find the element immediately before use —————
// Don't store WebElement — store the locator and re-find each time
By btnLocator = By.id("dynamicButton");
driver.findElement(btnLocator).click(); // always fresh reference

// — FIX 2: Wrap with retry logic —————
public void clickWithRetry(By locator, int maxAttempts) {
    for (int attempt = 1; attempt <= maxAttempts; attempt++) {
        try {
            driver.findElement(locator).click();
            return; // Success — exit
        } catch (StaleElementReferenceException e) {
            System.out.println("Stale attempt " + attempt + " — retrying");
            if (attempt == maxAttempts) throw e; // Give up after max tries
        }
    }
}

// — FIX 3: Use FluentWait with StaleElementReferenceException ignored —
FluentWait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(15))
    .pollingEvery(Duration.ofMillis(500))
    .ignoring(StaleElementReferenceException.class)
    .ignoring(NoSuchElementException.class);

wait.until(driver -> {
    driver.findElement(By.id("dynamicButton")).click();
    return true;
});
```

11.5 Your Complete Mastery Checklist

Track your progress — check each item as you complete it:

Foundation

- Explain how Selenium WebDriver works (the architecture chain)
- Set up a Maven project with all required dependencies
- Write a script that opens a browser, navigates, and quits cleanly

Locators

- Use all 8 locator strategies in real code
- Write relative XPath with attributes, text(), and contains()
- Use XPath axes: parent, ancestor, following-sibling, preceding-sibling
- Write CSS selectors with ID, class, attribute, nth-child, descendant
- Test locators in Chrome DevTools before writing code

Interactions

- Handle all form elements: input, textarea, Select dropdown, checkbox, radio
- Handle all three alert types: Alert, Confirm, Prompt
- Switch to iFrames and back to main content
- Handle multiple browser windows/tabs
- Use Actions class for hover, double-click, right-click, drag-and-drop
- Use keyboard shortcuts with Keys.chord()

Waits

- Understand WHY timing issues happen
- Use Explicit Wait with all major ExpectedConditions
- Use FluentWait with custom polling and exception ignoring
- Never use Thread.sleep() in production tests

Framework

- Build a complete POM framework: BasePage, page classes, BaseTest
- Write TestNG tests with all annotations (@BeforeMethod, @AfterMethod, etc.)
- Use all Assert methods including SoftAssert
- Create a @DataProvider and run data-driven tests
- Set up Extent Reports with screenshots on failure
- Read test configuration from config.properties

Advanced

- Run tests in headless Chrome mode
- Support multiple browsers via DriverFactory
- Set up and connect to Selenium Grid
- Write and run a Cucumber BDD feature file
- Configure GitHub Actions to run tests automatically on push

You are now a Selenium with Java Expert.

Practice each module, build real projects, ask questions — mastery comes from doing.

11.6 Handling Dynamic Content — AJAX & Single Page Apps

Modern web apps (React, Angular, Vue) update the DOM without full page reloads. This means driver.get() finishing does NOT mean the page content is loaded. Always use Explicit Wait for dynamic content.

Example: Waiting for React/Angular content to load

```
// — Waiting for Angular apps to finish rendering —————
// Angular apps have a special 'ng-scope' or '[_ngghost]' attribute when ready
// Use Explicit Wait for the actual data elements:

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(20));

// Navigate to page
driver.get("https://angular-app.example.com/products");

// Wait for product cards to appear (they are loaded via API call)
wait.until(ExpectedConditions.numberOfElementsToBeMoreThan(
    By.cssSelector(".product-card"), 0
));

List<WebElement> products = driver.findElements(By.cssSelector(".product-card"));
System.out.println("Products loaded: " + products.size());

// — Wait for a loading spinner to disappear —————
// Click something that triggers loading:
driver.findElement(By.id("loadDataBtn")).click();

// Wait for spinner to appear first (optional, confirms loading started)
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("spinner")));

// Then wait for spinner to disappear (loading complete)
wait.until(ExpectedConditions.invisibilityOfElementLocated(By.id("spinner")));

// NOW the data is ready
String result = driver.findElement(By.id("dataTable")).getText();

// — Execute JavaScript to check if jQuery AJAX is complete —————
// Useful for jQuery-heavy apps
wait.until(driver -> {
    return (Boolean) ((JavascriptExecutor) driver)
        .executeScript("return jQuery.active === 0;");
});
```

11.7 Running Tests with Maven from Command Line

```
# Run ALL tests:
mvn test

# Run a specific test class:
mvn test -Dtest=LoginTest

# Run a specific test method:
mvn test -Dtest=LoginTest#testValidLogin

# Run tests in a specific TestNG group:
mvn test -Dgroups=smoke

# Pass system properties to override config:
mvn test -Dbrowser=firefox -Dheadless=true

# Run with a specific testng.xml suite file:
mvn test -DsuiteXmlFile=testng.xml

# Skip tests (just build):
mvn package -DskipTests
```

```
# Clean + run all tests:
mvn clean test
```

11.8 Interview Questions & Answers for Selenium with Java

These are the most commonly asked Selenium interview questions. Study these after completing all modules.

Interview Question	Key Points to Cover in Your Answer
What is Selenium WebDriver? How is it different from Selenium IDE?	WebDriver: programmatic API for browser control, cross-browser, supports Java/Python etc. IDE: browser extension for record/playback only, limited and not suitable for real projects.
Explain the Selenium WebDriver architecture.	4 layers: Your Java code → Selenium API → HTTP/JSON → Browser Driver (e.g. chromedriver) → Browser. W3C WebDriver Protocol connects them.
What are the different locator strategies?	8 strategies: id, name, className, tagName, linkText, partialLinkText, cssSelector, xpath. Prefer id first, then CSS. XPath for complex relationships.
What is the difference between findElement and findElements?	findElement: returns single WebElement, throws NoSuchElementException if not found. findElements: returns List<WebElement>, returns empty list if none found.
What are the types of waits in Selenium?	Implicit Wait (global, DOM presence only), Explicit Wait (specific condition, recommended), Fluent Wait (custom polling + exception ignoring). Never Thread.sleep().
What is the difference between driver.close() and driver.quit()?	close(): closes only the current browser window. quit(): closes ALL windows AND kills the driver process. Always use quit() in teardown.
What is StaleElementReferenceException and how do you handle it?	Happens when DOM re-renders after element was found. Fix: re-find the element using the By locator instead of storing WebElement, or use FluentWait ignoring this exception.
What is the Page Object Model?	Design pattern where each page has a class containing its locators and action methods. Tests call page methods only. Benefits: maintainability, reusability, readability.
How do you handle alerts in Selenium?	driver.switchTo().alert() — returns Alert object. accept() for OK, dismiss() for Cancel, getText() for message, sendKeys() for prompt input.
How do you handle iframes?	driver.switchTo().frame(index/name/WebElement) to enter. driver.switchTo().defaultContent() to return to main page. driver.switchTo().parentFrame() for nested frames.
What is the Select class used for?	Handles HTML <select> dropdown elements. selectByVisibleText(), selectByValue(), selectByIndex(). getOptions() for all options. isMultiple() for multi-select.
What is JavascriptExecutor? When do you use it?	Interface to run JavaScript in the browser. Use when: normal click fails (element hidden), setting values on read-only fields,

	scrolling, reading page data, highlighting elements for debugging.
How do you run tests in parallel?	In testng.xml: parallel='methods' or 'tests' with thread-count. Use ThreadLocal<WebDriver> in DriverManager so each thread has its own browser instance.
What is the difference between hard assert and soft assert?	Hard Assert (Assert): stops test immediately on first failure. SoftAssert: collects all failures and reports them all at the end via assertAll(). Use SoftAssert when validating multiple independent things in one test.

